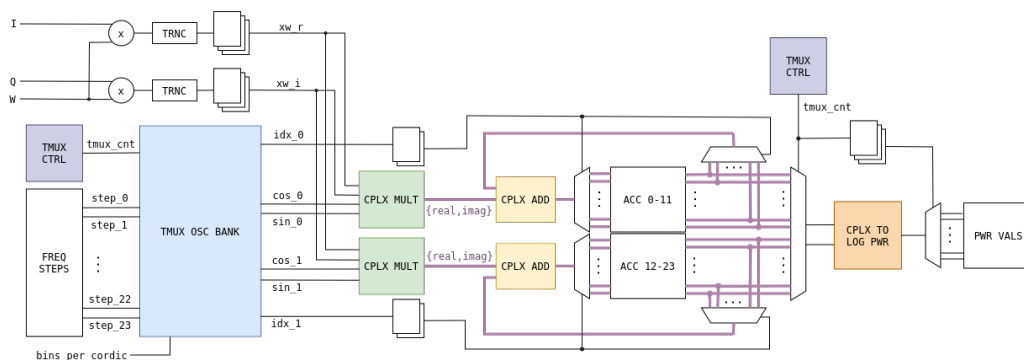


DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING

Fall Semester 2025

Real-Time Adaptive Data Compression for Ultrafast Ultrasound: FPGA-Based Streaming Bandwidth Estimation

Bachelor Thesis



Matteo Dietz
mdietz@student.ethz.ch

January 2026

Advisors: Federico Villani, villanif@ethz.ch
Andrea Cosettini, cosandre@ethz.ch

Professor: Prof. Dr. Luca Benini, lbenini@iis.ee.ethz.ch

Abstract

Modern medical ultrasound diagnostics increasingly demand higher transducer channel counts and sampling rates to improve image resolution, creating severe data throughput bottlenecks at the interface between acquisition hardware and host systems. This thesis presents a real-time, closed-loop adaptive data compression architecture for the ListenToLight platform that addresses these bandwidth constraints through weakly lossy, bandwidth-aware decimation. The proposed system employs a resource-efficient Streaming Sparse DFT algorithm implemented on a Xilinx Kintex UltraScale+ FPGA to continuously estimate the instantaneous signal bandwidth, enabling dynamic adjustment of the decimation factor to maintain Nyquist compliance while maximizing data reduction.

Validation using the PICMUS datasets demonstrates substantial performance gains: Compared to a naive worst-case decimation strategy based on transducer specifications, the adaptive system achieves additional compression ratios of $1.07\text{--}1.14\times$, representing 7–12% further bandwidth reduction. Critically, decimation safety exceeds 98.6% across all datasets, with in vivo scenarios achieving 99.2–99.6%, indicating negligible aliasing risk. Functional accuracy shows maximum edge mean absolute percentage error below 7.2% (0.09–0.15 MHz), confirming robust bandwidth tracking.

Hardware synthesis targeting a Kria K26 System-on-Module confirms remarkable resource efficiency and scalability. The complete design occupies only 5.6% of available logic resources, 2.5% of flip-flops, and 1% of DSP slices while consuming zero BRAM tiles, a critical advantage that enables linear scaling to high channel counts. This BRAM-free Sparse DFT architecture significantly outperforms industry-standard streaming FFT IP cores in resource utilization. The system operates at 375 MHz and is aimed at processing high-rate I/Q data streams with minimal latency, making it suitable for real-time deployment in multi-channel ultrasound platforms.

Acknowledgments

This thesis represents a significant milestone in my academic journey, providing me with the opportunity to dive deep into the intricacies of digital signal processing for ultrafast ultrasound imaging.

I would like to express my sincere gratitude to my supervisors, Federico Villani and Andrea Cosettini, for their mentorship during this project. Their combined knowledge, technical advice, and constructive feedback were invaluable in navigating the challenges of real-time data compression and shaping the final outcome of this work.

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Real-Time Adaptive Data Compression for Ultrafast Ultrasound: FPGA-Based Streaming Bandwidth Estimation

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

Dietz

First name(s):

Matteo

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Zürich, 05.01.2026

Signature(s)

M. Dietz

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).

Contents

List of Figures	vii
List of Tables	viii
1. Introduction	1
2. Background	5
2.1. Principles of Ultrasound Signal Processing	5
2.2. Digital Down Conversion	6
2.2.1. Sampling Ultrasound Data	6
2.2.2. I/Q Demodulation	6
2.2.3. Decimation	7
2.3. System Overview	7
2.3.1. The Analog Front-End	8
2.4. Spectral Analysis	9
2.4.1. Discrete Fourier Transform	9
2.4.2. Fast Fourier Transform	9
2.4.3. Windowing and Spectral Leakage	10
2.4.4. Equivalent Noise Bandwidth and Power Spectral Density	10
2.4.5. Time-Frequency Resolution and the Uncertainty Principle	11
2.5. Hardware Implementation Principles	12
2.5.1. CORDIC Algorithm	12
3. System and Algorithm Design	14
3.1. Real-Time Bandwidth Estimation	14
3.1.1. Sparse DFT Computation	14
3.1.2. Spectral Power Estimation and Bandwidth Detection	16
3.1.3. Memory Footprint	18
3.2. Adaptive Decimation Filter	20
3.2.1. Signal-to-Noise-and-Distortion Ratio Comparison	22
3.2.2. Signal-to-Quantization-Noise Ratio Comparison	23

Contents

3.3. Hardware Architecture Overview and Partitioning	26
4. Implementation	28
4.1. RTL Architecture	28
4.1.1. Top-Level Integration	28
4.1.2. Configuration and Control Interfaces	29
4.1.3. Spectral Power Estimation	32
4.1.4. Bandwidth Edge Detection	36
4.2. Verification Methodology	37
4.2.1. Simulation Framework	37
4.2.2. Stimuli Generation	38
4.2.3. Validation of the Virtual AFE	39
4.2.4. Golden Model	41
5. Results	43
5.1. Functional Accuracy	43
5.1.1. Error Metrics	43
5.1.2. Performance on Experimental Datasets	44
5.1.3. Performance on In Vivo Datasets	45
5.1.4. Inherent Bandwidth Overestimation and Safety Margin	46
5.2. Achieved Data Reduction	47
5.3. Resource Utilization	48
5.3.1. Comparison with Industry-Standard FFT IP Cores	50
5.3.2. Scalability Analysis	51
5.4. Current Limitations and Future Work	52
6. Conclusion and Future Work	53
A. Signal Processing Derivations	55
A.1. Further Details on I/Q Demodulation	55
A.2. On Signal-to-Quantization-Noise Ratios	56
B. Task Description	58
Bibliography	66

List of Figures

2.1. ListenToLight System Overview	8
3.1. Sparse DFT Bandwidth Estimation	18
3.2. Iterative Refinement Demonstration	19
3.3. Decimation Filter Performance Comparison Methodology	22
3.4. Comparative Analysis of Decimation Filter Performance	24
3.5. Hardware Partitioning	26
4.1. Spectral Power Estimator Architecture	31
4.2. Virtual AFE Validation	40
5.1. Bandwidth Overestimation for Gradual Spectra	46

List of Tables

3.1. Memory Footprint Comparison: Standard FFT vs. Sparse DFT	20
4.1. Virtual AFE Validation on the PICMUS experiments datasets	41
4.2. Virtual AFE Validation on the PICMUS in_vivo datasets	41
5.1. Functional accuracy evaluated on the PICMUS experiments datasets . .	44
5.2. Functional accuracy evaluated on the PICMUS in_vivo datasets	45
5.3. Data Reduction Performance	48
5.4. Complete Resource Utilization	49
5.5. Resource Utilization Comparison of Sparse DFT with FFT IP Cores . . .	50

Introduction

For decades, medical ultrasound has been a cornerstone of diagnostics imaging with broad clinical and preclinical applications, valued for its real-time capabilities, non-ionizing nature and relatively low cost [1]. Continuous innovation, from new imaging and processing techniques to portable hardware and sophisticated software, ensures its central role in fields including oncology, cardiology, brain imaging, obstetrics [2], emergency medicine and many more. As technology advances, the demand for higher resolution and more sophisticated imaging modes constantly pushes system designers to capture more data with an ever-increasing number of channels and higher sampling rates. The management of these massive datastreams strains the limits of acquisition, real-time processing and transmission in ultrasound-based imaging modalities. Addressing this data-rate bottleneck is therefore a critical step in enabling future innovations.

Data-rate reduction, respectively data compression in general, can be approached from a variety of different angles. To systematically evaluate candidate techniques for high-performance ultrasound research platforms, we categorize compression methods in three different classes of fidelity preservation: **strongly lossy**, **lossless**, and **weakly lossy** approaches. Each category presents distinct trade-offs between compression efficiency, signal integrity, and implementation complexity.

Prior work on strongly lossy methods has shown that very high compression ratios can be achieved by transforming the ultrasound signal into another domain and intelligently discarding information that is deemed less important, respectively considered to be noise. These techniques operate under strong assumptions about the underlying signal structure, most critically, the assumption of sparsity, which means that the signal contains either large regions of empty space or simple structure that can be mathematically reconstructed from a heavily undersampled representation.

For instance, Drori et al. [1] have proposed a novel beamforming algorithm in the frequency domain that is equivalent to time domain DAS. Combined with compressed

1. Introduction

sensing techniques and sub-Nyquist sampling, this approach allows them to filter and massively compress the ultrasound data by exploiting the finite rate of innovation (FRI) structure of the beamformed data.

Similar techniques based on the Discrete Wavelet Transform (DWT) can efficiently denoise and significantly reduce the dimensionality of medical imaging data by thresholding small wavelet coefficients. In the landscape of lossy compression algorithms, the DWT shows distinguished potential due to its ability to simultaneously represent information in time and frequency domains while addressing noisiness, excessive storage requirements, and low information content [3].

While these strongly lossy techniques are powerful and represent a compelling field of research, they are fundamentally ill-suited for the objectives of a high-performance, flexible research platform like the *ListenToLight* system used in this thesis. This platform is characterised by its high-bandwidth architecture, capable of transferring 80Gbit/s to the host PC. The primary goal of such a system is to provide highest possible quality data without compromising signal fidelity. In the context of *ListenToLight*, three key issues make aggressive, lossy compression undesirable:

- **Unacceptable Loss of Fidelity:** Any lossy step at the acquisition stage can permanently degrade the ground truth data, which defies the purpose of a high-end research system.
- **Violation of the Open Platform Philosophy:** Approaches like Frequency Domain Beamforming force the downstream user into a specific reconstruction method, restricting the freedom of researchers to develop and test their own novel algorithms in software.
- **Excessive Compression:** With the ability to handle high data rates, the significant trade-off in signal quality for massive compression factors is difficult to motivate.

At the opposite end of the spectrum, truly lossless compression techniques such as Huffman coding, Lempel-Ziv-Welch (LZW), arithmetic coding, and similar entropy-based algorithms offer the compelling property of mathematical reversibility with zero degradation of signal fidelity. These methods operate by analyzing patterns in the digitized bit stream, constructing dictionaries or variable-length codes to exploit statistical redundancy in the data representation itself.

Recent work by Kim et al. [4] demonstrates a real-time FPGA implementation of lossless compression for ultrasound RF data, achieving compression ratios of 20–30%. While this represents a notable engineering achievement, it simultaneously exposes the fundamental limitations of truly lossless approaches for high-throughput ultrasound systems:

- **Insufficient Compression:** The achievable compression ratios, typically on the order of 20–30%, are inherently constrained by the statistical properties of raw ultrasound data. The high entropy ultrasound signals leave limited redundancy to

1. Introduction

exploit, rendering strictly lossless schemes insufficient for the substantial data-rate reductions required by scalable, multi-channel systems.

- **Inherently Computation-Driven:** Lossless encoding-based compression approaches are inherently computation-driven and must therefore be implemented directly on the FPGA. This consumes additional FPGA resources that could otherwise be dedicated to signal processing or channel scaling.
- **Too Late in the Processing Chain:** Most critically, lossless compression operates after the AFE has already transmitted the uncompressed data rate to the FPGA. As a result, although compression may reduce the data volume forwarded to the host system, it provides no relief at the AFE-FPGA interface which ultimately limits scalability.

For these reasons, truly lossless compression, despite its theoretical elegance, represents an impractical solution for the data-rate challenges of high-channel-count ultrasound systems such as the *ListenToLight* platform.

Between the extremes of strongly lossy and truly lossless compression lies a third paradigm: weakly lossy methods that exploit the inherent band-limited nature of ultrasound physics. Rather than discarding signal components (strongly lossy) or encoding bit patterns (lossless), these techniques discard frequency content that lies outside the bandwidth of the physical ultrasound transducer. One method for weakly lossy compression is quadrature demodulation followed by decimation. Critically, by strictly adhering to the Nyquist-Shannon sampling theorem, maintaining a sampling rate at least twice the signal bandwidth, this approach ensures that the discarded samples represent only empty spectrum or noise, not signal content.

This weakly lossy paradigm occupies a uniquely advantageous position:

- **Minimal Information Loss:** Unlike strongly lossy methods, decimation does not discard actual signal components or make aggressive sparsity assumptions. The information loss is statistically insignificant, limited to out-of-band noise and quantization effects that do not affect reconstruction fidelity.
- **Hardware Efficiency:** Unlike truly lossless algorithms, decimation filters are standard blocks available directly in modern AFE architectures. This allows data reduction at the source, before transmission across the AFE-FPGA link.

The effectiveness of weakly lossy compression through decimation is fundamentally limited by the signal bandwidth itself. For a fixed decimation factor, the approach assumes a uniform maximum bandwidth across all imaging scenarios. In practice, ultrasound signals exhibit massive variability in spectral content depending on target composition, imaging depth, transducer characteristics, and acquisition parameters. A conservative fixed decimation factor that accommodates worst-case bandwidth results in significant oversampling for the majority of imaging conditions where the actual signal bandwidth is substantially narrower.

1. Introduction

This observation motivates the central contribution of this thesis: **adaptive decimation based on real-time bandwidth estimation**. By dynamically adjusting the decimation factor to the instantaneous signal bandwidth while maintaining strict Nyquist compliance, the system can achieve substantial data reduction for multi-channel ultrasound platforms.

Prior work has explored adaptive decimation for medical ultrasound imaging. Kim et al. [5] and Kang et al. [6] proposed dynamic decimation filter architectures using polyphase multiply-accumulate (MAC) structures implemented on FPGAs, capable of adjusting both decimation ratio and filter coefficients in real-time. However, these approaches require dedicated FPGA hardware resources for implementing the decimation filters, whereas this work aims to leverage the programmable decimation capabilities already available in modern AFE hardware, enabling data reduction at the source while minimizing additional logic requirements.

This thesis proposes a closed-loop adaptive decimation control system that consists of three main subsystems: an FPGA-based streaming bandwidth estimation module that continuously analyzes the spectral content of incoming I/Q data, an APU-based configuration and control layer that computes safe decimation parameters using the spectral content found by the FPGA, and an AFE hardware decimator that dynamically adjusts its decimation rate in response to measured bandwidth. This closed-loop architecture enables the acquisition hardware to intelligently adapt to real-time signal characteristics, maximizing data reduction while guaranteeing aliasing-free capture.

The approach provides several key advantages: it is weakly lossy by design; it addresses both the AFE-to-FPGA and FPGA-to-Host bandwidth constraints; it preserves the open platform philosophy and signal fidelity without forcing downstream users into specific reconstruction methods; and it remains robust and flexible across various transducers and imaging scenarios, requiring only that signals are band-limited, which is a property guaranteed by the physics of ultrasound transduction and propagation. This work represents a system-level contribution to the *ListenToLight* platform, proposing a hardware-integrated solution for adaptive data reduction in high-throughput ultrasound research systems.

Background

2.1. Principles of Ultrasound Signal Processing

The generation of ultrasound (US) signals is based on the piezoelectric effect. When electrical energy is applied to piezoelectric crystals inside the US probe, it is converted into acoustic energy and emitted as mechanical waves into the tissue. Upon encountering tissue interfaces with sudden changes in acoustic impedance, these waves are reflected back towards the probe, inside of which they converted back into electrical signals.

Right after reception, these electrical signals are subject to various analog signal processing steps inside the analog front-end (AFE). This analog preprocessing is necessary to address physical phenomena such as attenuation [7]. Attenuation is caused by several factors, including the following:

- Refraction, which describes the deflection of the US wave away from its original direction when it crosses an interface between two media with different sound speeds at an oblique angle.
- Scattering, which occurs when ultrasound encounters structures comparable to or smaller than the wavelength, causing the energy to be redirected into many direction.
- Absorption, which is the conversion of acoustic energy into heat within the tissue.

Diagnostic ultrasound operates using pulsed ultrasound, where short-duration bursts of high-frequency acoustic energy are transmitted into the tissue. The received radio-frequency (RF) signal, which is digitized by Analog-to-Digital Converters (ADCs), can be modeled as a bandpass signal centered around the transducer center frequency f_c , which is defined as the frequency at which the transducer exhibits its maximum transmit–receive sensitivity. The spectral content of the pulse occupies a finite range of

2. Background

frequencies, characterized by the transducer bandwidth, which is typically specified as the frequency range over which the transducer response remains within a given level (commonly -6dB) of its peak.

2.2. Digital Down Conversion

Once the analog signal has been digitized by the ADCs, we are left with the resulting RF signal $x_{RF}(t)$. This Signal contains the intact frequency information but mostly at a data rate that is unnecessarily high for storage and further processing. Digital Down Conversion (DDC) is the technique used to reduce this data volume while preserving its essential information.

2.2.1. Sampling Ultrasound Data

The received RF signal from an ultrasound transducer is inherently a band-pass signal. While the sampling rate might be very high (e.g., 80 MHz) to satisfy the Nyquist theorem for the highest frequency component, the actual information of interest resides within a much narrower bandwidth. Typically, the relative bandwidth of a transducer is 50-70% of its center frequency [8].

For example, a transducer with center frequency of 10 MHz and a 70% bandwidth will have all relevant frequency content approximately between 6.5 and 13.5 MHz. While a sampling rate of 80 MHz can capture all frequency information, it results in a significant data surplus.

This surplus becomes a critical bottleneck in modern US systems, which may feature a large number of channels (e.g., 128 or 256), all simultaneously producing data at 80 MHz. The resulting aggregate data rate exceeds the I/O bandwidth of even high-performance interconnects, making it impossible to transfer the raw RF data to a host PC for downstream processing. Although the sampling rate is well above the Nyquist rate, which allows for simple decimation (e.g., by a factor of 2 to reach an effective sampling rate of 40 MHz), but a significantly more resource efficient method is to use I/Q demodulation.

2.2.2. I/Q Demodulation

I/Q demodulation, which also called quadrature mixing, is a process that shifts the frequency spectrum of the RF signal down to baseband and converts the real-valued RF data into a complex signal, consisting of the In-Phase (I) and Quadrature-Phase (Q) components. This allows the system to isolate the bandwidth of interest and discard redundant data. I/Q demodulation consists of following steps:

2. Background

1. Down-mixing: The real-valued RF signal $x_{rf}(t)$ is multiplied by a complex sinusoid signal to shift the spectrum. The resulting complex signal $\tilde{x}_{IQ}(t)$ is given by:

$$\tilde{x}_{IQ}(t) = x_{RF}(t) \cdot e^{-j2\pi f_{demod}t}, \quad (2.1)$$

where f_{demod} is the demodulation frequency, which is typically close to the center frequency of the transducer. Using Euler's formula, this operation is equivalent to mixing the RF signal with two sinusoidal waves with a phase shift of $\pi/2$ [8].

$$e^{-j\omega t} = \cos(\omega t) - j \cdot \sin(\omega t) \quad (2.2)$$

2. Low-pass filtering: After mixing, the signal spectrum is shifted to baseband. A low-pass filter is applied to remove any high-frequency components and any noise outside of the bandwidth of interest. This step essentially isolates the relevant signal energy and gives us the final $x_{IQ}(t)$.

A more mathematical description of I/Q demodulation can be found in Appendix A.1.

2.2.3. Decimation

After shifting the signal to baseband and low-pass filtering, the maximum frequency component of the IQ signal is significantly lower than that of the original RF signal. According to the Nyquist theorem, the sampling frequency can now be reduced without any loss of information.

Decimation is the process of reducing the sampling rate to match this new, narrower bandwidth. For a complex signal, the required sampling rate is determined by the complex bandwidth. Coming back to the example in Section 2.2.1, if the original 80 MHz signal is I/Q demodulated and low-pass filtered, it will have all of its frequency components between -3.5 and 3.5 MHz. The sampling rate could theoretically be reduced to 7 MHz, which is a significant improvement to the scenario in Section 2.2.1.

2.3. System Overview

As mentioned in the introduction, this work is embedded within the context of the *ListenToLight* project at IIS at ETH Zurich. *ListenToLight* aims to develop a novel optoacoustic imaging system capable of reaching sampling rates of up to 125 Mega Samples per second (MSPS) with 256 channels. While high sampling rates and high channel counts are necessary to improve image quality, they create a significant bottleneck in data transmission and storage.

The overall system architecture is depicted in Figure 2.1. The system interfaces 256 ultrasound channels via 16 Pulser T/R switches to the Analog Front-End (AFE). The

2. Background

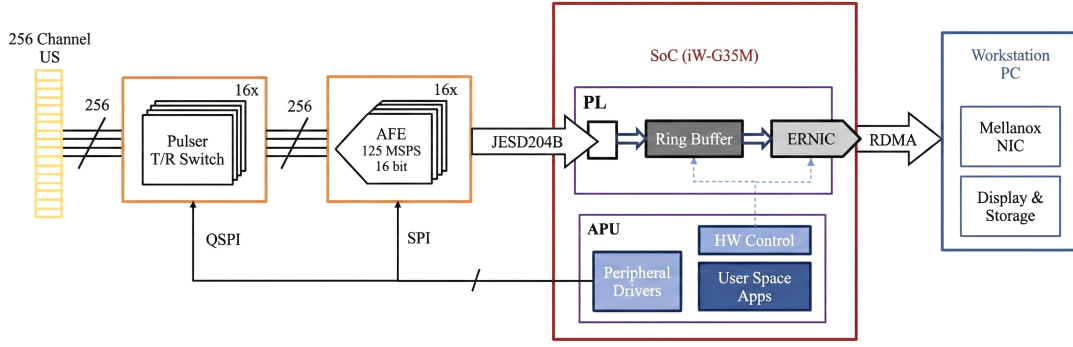


Figure 2.1.: ListenToLight System Overview

digitized data is transmitted via a JESD204B link to the Programmable Logic (PL) of the ZU90EG SoC, which consists of a Xilinx Kintex UltraScale+ FPGA. Inside the PL, the data stream is managed by a Ring Buffer and an Embedded RDMA Network Interface Controller (ERNIC), which points to an RDMA link connecting the SoC to the Host Workstation for downstream processing (beamforming) and storage. The hardware control and peripheral drivers are managed by the SoC's Application Processing Unit (APU).

2.3.1. The Analog Front-End

The core of the signal acquisition chain is the Texas Instruments AFE58JD48 [9]. This is a high-end 16-channel AFE designed specifically for ultrasound applications, offering 16-bit output resolution at 125 MSPS. Beyond various analog signal processing steps and analog-to-digital conversion, the AFE features a sophisticated digital processing pipeline relevant to this thesis.

After the initial ADC stage, the signal passes through digital blocks for offset correction and high-pass filtering before reaching the Digital Demodulator. This block performs I/Q demodulation as described in Section 2.2.2, required for the subsequent processing stages. The resulting I/Q streams are passed through a programmable decimation filter and multiplexed (compressed) onto the serial JESD204B interface for transmission to the FPGA.

We are doing this work specifically to optimize the data transmissions for this system.

2. Background

2.4. Spectral Analysis

2.4.1. Discrete Fourier Transform

The Discrete Fourier Transform (DFT) analyzes the frequency content of a discrete signal $x[n]$ of finite length N . According to the standard definition, the transformation of the time-domain signal into its frequency-domain representation $X[k]$ is given by

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad (2.3)$$

where k represents the discrete frequency bin index ($0 \leq k < N$).

In physical system with a sampling frequency f_s , the frequency f_k corresponding to bin index k is defined as:

$$f_k = k \cdot \Delta f = k \cdot \frac{f_s}{N} \implies k = \frac{f_k \cdot N}{f_s} \quad (2.4)$$

Substituting this back into the standard form of the DFT gives us the following alternative representation, which we will use in this report from now on:

$$X(f_k) = \sum_{n=0}^{N-1} x[n] e^{-j2\pi f_k n / f_s} \quad (2.5)$$

2.4.2. Fast Fourier Transform

The Fast Fourier Transform (FFT) is an algorithm designed to efficiently calculate the DFT. To be specific, the computational complexity of a naive N -point DFT is $\mathcal{O}(N^2)$ operations. Using a Divide-and-Conquer approach, the FFT reduces this to $\mathcal{O}(N \log N)$ for signals of length $N = 2^M$, for $M \in \mathbb{N}$.

The main idea is to recursively partition the N -point DFT into two $\frac{N}{2}$ -point DFTs, one for the even and one for the odd indices, until we are left with only DFTs of length 2. Let $\omega_N := e^{-j2\pi/N}$, then one such step looks as follows:

$$X[k] = \sum_{n=0}^{N-1} x[n] \omega_N^{kn} = \underbrace{\sum_{l=0}^{\frac{N}{2}-1} x[2l] \omega_{\frac{N}{2}}^{kl}}_{=:G[k]} + \omega_N^k \underbrace{\sum_{l=0}^{\frac{N}{2}-1} x[2l+1] \omega_{\frac{N}{2}}^{kl}}_{=:U[k]} \quad (2.6)$$

$G[k]$ and $U[k]$ are $\frac{N}{2}$ -point DFTs. Using this principle, we can recursively divide the lengths of the DFTs by two until we are left with $\frac{N}{2}$ DFTs of length 2, in which case the DFT simplifies to a simple addition and subtraction.

2. Background

2.4.3. Windowing and Spectral Leakage

The DFT processes a finite sequence $x[n]$ of N samples, but it implicitly assumes that the signal repeats infinitely after the observed interval. I.e., $x[n + N] = x[n] \quad \forall n \in \mathbb{Z}$.

If $x[0] \neq x[N - 1]$, this creates a discontinuity at the boundaries of the periodic extension. In the frequency domain, this sharp discontinuity requires infinite high-frequency components to be represented correctly, causing the signal energy to leak into adjacent frequency bins. Hence, this phenomenon is called Spectral Leakage. To mitigate these undesirable effects, window functions are used [10]. The expression for the windowed DFT then becomes

$$X(f_k) = \sum_{n=0}^{N-1} x[n] w[n] e^{-j2\pi f_k n / f_s}, \quad (2.7)$$

which is also called the discrete Short-Time Fourier Transform (STFT) of $x[n]$.

By multiplying the time domain signal $x[n]$ with a tapered window $w[n]$ that smoothly approaches zero at the edges ($n = 0$ and $n = N - 1$), the boundary discontinuity is eliminated. This significantly reduces spectral leakage.

However, the choice of window function determines how much of the signal and the surrounding noise is accumulated into each frequency bin [11][12]. Consequently, the amplitude estimate at a given frequency bin is influenced by the noise power integrated over the effective bandwidth of the selected window function. When calculating the Power Spectral Density (PSD) of a signal, we need to account for this fact.

2.4.4. Equivalent Noise Bandwidth and Power Spectral Density

The Equivalent Noise Bandwidth (ENBW) is the bandwidth of an ideal bandpass filter H that accumulates the same amount of noise power as a window function W . Both the ideal filter H and the window W are assumed to have the same peak gain. The total noise power for white input noise with a PSD of N_0 accumulated by a window is given by

$$P_{W,noise} = \int_{-f_s/2}^{f_s/2} N_0 |W(f)|^2 df = \frac{N_0}{T} \sum_{n=0}^{N-1} |w[n]|^2, \quad (2.8)$$

where we used Parseval's Theorem to calculate the power in time domain using the discrete window coefficients $w[n]$. The peak power gain of the window occurs at DC ($f = 0$).

$$G_{peak} = W^2(0) = \left| \sum_{n=0}^{N-1} w[n] \right|^2 \quad (2.9)$$

2. Background

Since the ideal filter has a rectangular spectrum with bandwidth $ENBW_{Hz}$ and gain G_{peak} , its accumulated noise power is given by:

$$P_{H,noise} = N_0 \cdot G_{peak} \cdot ENBW_{Hz} \quad (2.10)$$

To find the Equivalent Noise Bandwidth, we equate the noise power of the window $P_{W,noise}$ and the noise power of the ideal filter $P_{H,noise}$ and solve for $ENBW_{Hz}$. We get:

$$ENBW_{Hz} = \frac{1}{T} \frac{\sum_{n=0}^{N-1} |w[n]|^2}{\left| \sum_{n=0}^{N-1} w[n] \right|^2} = f_s \frac{\sum_{n=0}^{N-1} |w[n]|^2}{\left| \sum_{n=0}^{N-1} w[n] \right|^2} \quad (2.11)$$

The definition and derivation of the Equivalent Noise Bandwidth presented in this subsection are derived following the approach described by Harris. For a comprehensive discussion of window functions and their effects on spectral analysis, refer to *“On the Use of Window Functions for Harmonic Analysis with the Discrete Fourier Transform”* [11].

We need this expression for the ENBW to connect the two primary metrics that are used to quantify spectral energy: the Power Spectrum (PS) and the Power Spectral Density (PSD). While related, they serve different analytical purposes. The Power Spectrum estimates the total power of discrete frequency components (e.g., single tones), whereas the PSD estimates the power distribution of broadband signals (e.g., noise) normalized to a per-Hertz basis. The two metrics are linked by the Equivalent Noise Bandwidth, which acts as the normalization factor for the effective bandwidth of the window:

$$PSD(f_k) = \frac{PS(f_k)}{ENBW_{Hz}} \quad (2.12)$$

Given the discrete Power Spectrum calculated from the windowed DFT, we can find the PSD as follows:

$$PS(f_k) = \frac{|X(f_k)|^2}{\left| \sum_{n=0}^{N-1} w[n] \right|^2} \implies PSD(f_k) = \frac{|X(f_k)|^2}{f_s \sum_{n=0}^{N-1} |w[n]|^2} \quad (2.13)$$

2.4.5. Time-Frequency Resolution and the Uncertainty Principle

Another important metric to consider when looking at windowed DFTs is the window length N . The selection of this window length imposes a fundamental trade-off between temporal resolution and frequency resolution, a constraint governed by the Uncertainty Principle.

In Fourier analysis, this principle states that a signal cannot be localized with arbitrary precision in both time and frequency domain simultaneously [13]. In the context of

2. Background

windowed DFTs, this trade-off arises from the duality between multiplication in the time domain and convolution in the frequency domain. Applying a finite-length window corresponds to multiplying the signal in time, which in turn results in a convolution of the signal spectrum with the Fourier transform of the window.

Qualitatively, a function that is narrow in time has a wide Fourier transform, while a function that is wide in time has a narrow Fourier transform [14]. In either domain, increased width corresponds to a broader distribution of energy, which implies increased uncertainty. This means, reducing uncertainty in one domain necessarily increases uncertainty in the other.

As a consequence, a short window provides good temporal localization but leads to reduced frequency resolution due to increased spectral spreading. Meanwhile, a long window improves frequency resolution at the expense of temporal resolution [15]. This trade-off is fundamental and reflects the time-frequency uncertainty principle, rather than a limitation of the DFT or a particular window function.

2.5. Hardware Implementation Principles

2.5.1. CORDIC Algorithm

This section provides a brief overview of the relevant parts of the Coordinate Rotation Digital Computer (CORDIC) algorithm as detailed in the Xilinx CORDIC v6.0 LogiCORE IP Product Guide (PG105) [16].

CORDIC is an iterative algorithm used to compute vector rotations or translations. In the context of the DFT, it serves as a highly efficient method for generating the complex reference sinusoids $e^{-j2\pi f_k n / f_s}$, without the need for additional hardware multipliers or memory.

The fundamental operation of the CORDIC algorithm is the rotation of a vector (X, Y) by an angle θ to produce a new vector (X', Y') .

$$\begin{aligned} X' &= X \cdot \cos \theta - Y \cdot \sin \theta \\ Y' &= X \cdot \sin \theta + Y \cdot \cos \theta \\ \theta' &= 0, \end{aligned} \tag{2.14}$$

Instead of performing the rotation directly, which requires trigonometric functions, the algorithm decomposes the rotation angle θ into a sequence of n progressively smaller micro-rotations. The angle of the i -th micro-rotation is defined as $\arctan(2^{-i})$.

2. Background

By restricting the micro-rotations to these specific angles, the rotation operation simplifies to a series of shifts and additions

$$\begin{aligned} X_{i+1} &= X_i - \sigma_i \cdot Y_i \cdot 2^{-i} \\ Y_{i+1} &= Y_i + \sigma_i \cdot X_i \cdot 2^{-i} \\ \theta_{i+1} &= \theta_i + \sigma_i \cdot \arctan\left(2^{-i}\right), \end{aligned} \tag{2.15}$$

where θ represents the residual angle accumulator, and $\sigma_i = \pm 1$ determines the direction of rotation to drive θ towards zero. After n iterations, the vector is rotated by the desired angle θ . To generate the complex oscillator values $e^{-j2\pi f_k n / f_s}$ for the DFT, the algorithm is initialized with the vector $(1,0)$ and rotated by the phase accumulator value $\theta[n] = 2\pi \frac{f_k}{f_s} n$.

While CORDIC blocks avoid multipliers, they still suffer from quantization noise from two different sources:

- **Input Quantization Error:** This refers to the noise that is inherent to the finite width of the input vector and phase angle. For vector rotation, a quantization error of ± 0.5 LSB on the inputs propagates directly to the output components X' and Y' .
- **Internal Precision Error:** This corresponds to the error introduced during the finite precision shift-add iterations which contain some truncations.

Since the internal precision is usually (and mostly even automatically) set sufficiently high, the output quantization error is dominated by the input bit-width. Hence, for high-precision spectral analysis, maximizing the phase width is critical to minimizing accuracy.

System and Algorithm Design

To achieve optimal data compression within the Analog Front-End (AFE), the system must minimize the data rate of the incoming I/Q data while preserving signal integrity. As established in the Chapter 2, the fundamental limit for lossless data reduction is determined by the signal's occupied bandwidth. This leads to the key idea of this work:

If the precise bandwidth edges of the received signal are known, the system can dynamically maximize the decimation factor M , reducing the sampling rate to the lowest permissible value satisfying the Nyquist theorem.

$$f_{s,dec} \geq 2 \cdot f_{max} \implies M \leq \left\lfloor \frac{f_{s,in}}{2 \cdot f_{max}} \right\rfloor \quad (3.1)$$

The bandwidth of ultrasound signals is not static. It varies with transducer characteristics, transmit center frequency, and tissue-specific attenuation. Consequently, relying on a fixed, worst-case bandwidth estimate results in suboptimal compression. To address this challenge, a Real-Time Streaming Sparse DFT architecture is presented in Section 3.1. This system estimates the signal bandwidth on-the-fly without requiring the large memory buffers associated with traditional Fast Fourier Transforms (FFT).

3.1. Real-Time Bandwidth Estimation

3.1.1. Sparse DFT Computation

Standard spectral analysis methods, such as the FFT, require buffering an entire window of N samples before computation can begin. This introduces significant latency and

3. System and Algorithm Design

high memory utilization, especially as the number of channels that need to be analyzed increases. Due to this memory bottleneck, we employ a streaming approach that computes the DFT only for a sparse set S of selected frequency bins. This aligns well with our goal, since our application requires analyzing specific bandwidth edges rather than the entire spectrum.

As described in Section 2.4, the N -point windowed DFT for a specific frequency bin f_k is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot h[n] \cdot e^{-j2\pi f_k n / f_s}, \quad (3.2)$$

where $x[n]$ is the input sample, $h[n]$ is the window function (e.g., Hann), and f_s is the sampling rate. In our streaming architecture, this summation is implemented as a cumulative update. For each frequency f_k in our set of interest S , we maintain a complex accumulator A_k . As each new sample arrives, its contribution is computed and added to the accumulator:

$$A_k[n] = A_k[n-1] + x[n] \cdot h[n] \cdot e^{-j2\pi f_k n / f_s} \quad (3.3)$$

After N samples have been processed, the value of A_k corresponds exactly to the DFT component $X[k]$.

We could utilize the same recursive strategy to calculate the complex oscillator values efficiently in hardware. I.e., a phasor W_k is maintained for each bin and updated iteratively using a constant complex rotation factor E_k :

$$W_k[n] = W_k[n-1] \cdot E_k, \quad \text{where } E_k = e^{-j2\pi f_k / f_s} \quad (3.4)$$

However, recursive phasor multiplication inherently suffers from the issue of numerical drift, that accumulates over the length of the window. For the first few samples, the complex oscillator value is accurate, but the longer the window, the bigger the accumulated error is going to get towards later stages of the window.

To address this issue, a phase accumulator combined with a CORDIC engine is used in this architecture. Instead of iteratively multiplying complex numbers, we iteratively update a real phase argument, which is numerically much more stable and drift-free.

For each frequency bin f_k , we maintain a phase accumulator θ_k . At each time step n , the phase is incremented by a constant frequency bin dependent step $\Delta\theta_k$:

$$\theta_k[n] = \theta_k[n-1] + \Delta\theta_k, \quad \text{where } \Delta\theta_k = -2\pi f_k / f_s \quad (3.5)$$

$\theta_k[0]$ is initialized as 0. After n steps, the phase for bin f_k has the value $-2\pi f_k n / f_s$, which corresponds to the phase of the complex oscillator value that needs to be multiplied with the windowed sample $x[n] \cdot h[n]$. The accumulated phase $\theta_k[n]$ is then passed to a

3. System and Algorithm Design

CORDIC core that calculates the sine and cosine values by performing a series of bit-shift and add operations as described in Section 2.5.1.

$$W_k[n] = \cos(\theta_k[n]) + j \cdot \sin(\theta_k[n]) = e^{j\theta_k[n]} = e^{-j2\pi f_k n / f_s} \quad (3.6)$$

This generated phasor is then used in the accumulator update rule as before:

$$A_k \leftarrow A_k + x[n] \cdot h[n] \cdot W_k[n] \quad (3.7)$$

This approach decouples the resource utilization and specifically the memory complexity from the window length N . The memory footprint is proportional only to the number of calculated bins $K = |S|$. For a sparse set where $K \ll N$, this yields drastic savings compared to a full FFT, where all N samples need to get buffered.

Within a frame, the set of frequency bins of interest is as follows:

S_{coarse} is a very coarse set of points distributed evenly between $\pm f_c$, where f_c corresponds to the transducer center frequency. This set of frequency bins contains only a few points and acts as a coarse drift guard.

S_{left} , S_{right} are edge focused fine clusters that are initially centered around $\pm f_c/2$ because that is where we would approximately expect the bandwidth crossing to happen according to the transducer specification. If this initial heuristic is wrong, we still have the bins in S_{coarse} that tell us whether our initial guess was too high or too low. The total set of frequency bins is then given by:

$$S = S_{coarse} \cup S_{left} \cup S_{right} \quad (3.8)$$

An example using 24 bins can be seen in Figure 3.1, where the black points correspond to the set S .

3.1.2. Spectral Power Estimation and Bandwidth Detection

Once the sparse spectrum $\{A_k\}_{k \in S}$ has been computed, the resulting complex accumulators are passed to the analysis stage to determine the signal's occupied bandwidth.

First, the power spectrum is estimated for each calculated bin. The squared magnitude of the complex accumulator is computed and converted to a logarithmic scale to facilitate thresholding independent of the input signal level:

$$P_k^{\text{dB}} = 10 \cdot \log_{10}(|A_k|^2) = 10 \cdot \log_{10}(\mathcal{R}\{A_k\}^2 + \mathcal{I}\{A_k\}^2) \quad (3.9)$$

The algorithm then identifies the peak power value, P_{max} , within the set of interest. An absolute power threshold, P_{th} , is established by applying a configurable drop (e.g., -30 dB) relative to this peak:

$$P_{th} = P_{max} - P_{drop} \quad (3.10)$$

3. System and Algorithm Design

To locate the bandwidth edges, the power spectrum is scanned starting from the center frequency ($f = 0$) and moving outwards. For the lower bandwidth edge, the search proceeds towards negative frequencies to identify two adjacent bins, f_1 and f_2 , where the power crosses the threshold:

$$P(f_1) \leq P_{th} < P(f_2) \quad (3.11)$$

A similar bandwidth search is performed towards positive frequencies to locate the upper edge.

Because the frequency bins are discrete, the true threshold crossing likely lies between the identified points. To recover this precision, linear interpolation is performed in the log-power domain:

$$f^* = f_1 + (f_2 - f_1) \cdot \frac{P_{th} - P(f_1)}{P(f_2) - P(f_1)} \quad (3.12)$$

This calculation yields the final fine-tuned estimates for the left and right bandwidth edges, f_{left}^* and f_{right}^* .

The complete execution of this bandwidth estimation algorithm is visualized in Figure 3.1. This demonstration utilizes experimental ultrasound data from the PICMUS contrast speckle dataset [17], which was upsampled to match the system's 125 MHz ADC rate, I/Q demodulated, and decimated by a factor of 4. The analysis was performed on a specific signal segment (Channel 37, Window # 34) using a window length of $N = 256$ and a sparse set of $|S| = 24$ frequency bins. For validation, the sparse spectral power is plotted against a ground truth Power Spectral Density (PSD) computed using Welch's method. Notably, explicit correction for the window's ENBW is rendered unnecessary by the peak normalization process. Since the ENBW acts as a multiplicative scaling factor on the linear power spectrum as described in Section 2.4.4, it manifests as a constant additive offset in the logarithmic domain. Consequently, when normalizing the spectrum relative to its peak, this offset cancels out completely:

$$PSD_{norm}^{dB} = \left(P_{meas}^{dB} - 10 \log_{10} ENBW \right) - \left(P_{peak}^{dB} - 10 \log_{10} ENBW \right) = P_{meas}^{dB} - P_{peak}^{dB} \quad (3.13)$$

The frequency resolution of the estimated bandwidth is fundamentally determined by the bin spacing within the set S . The upper bound for the frequency resolution is equivalent to that of a standard FFT of the same length N , provided that the bins in the fine edge clusters (S_{left} and S_{right}) are chosen to be adjacent (i.e., spaced by f_s/N). The effectiveness of such dense spacing depends on the accuracy of the initial coarse bandwidth estimate used to position these clusters.

The portability and low memory footprint of this algorithm allow for an efficient iterative refinement strategy. Over subsequent frames, the system can use the detected edges f_{left}^* and f_{right}^* to re-center the fine search clusters and potentially even increase the window length N for a higher frequency resolution. This "zoom-in" approach enables

3. System and Algorithm Design

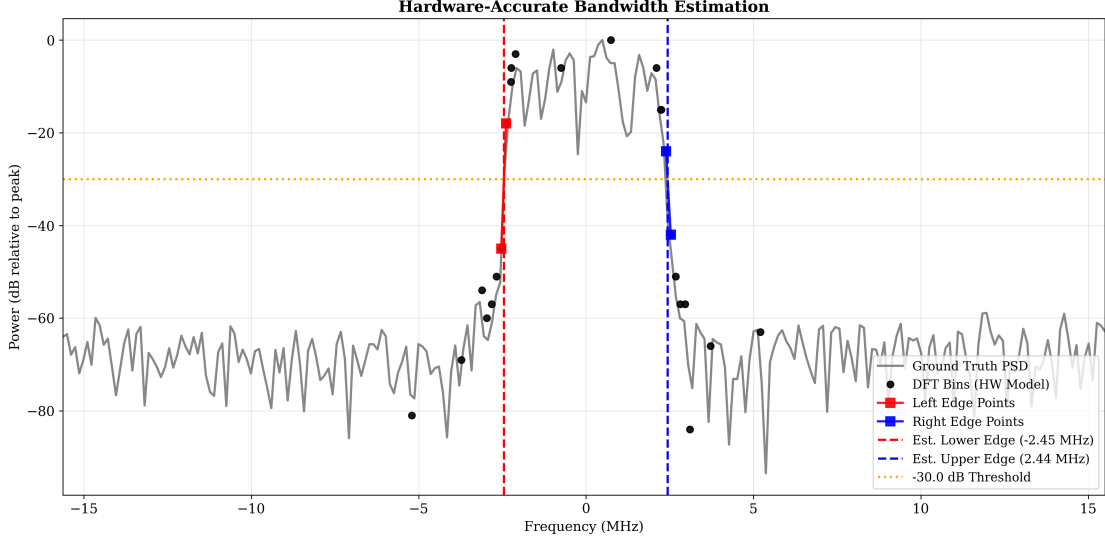


Figure 3.1: Illustration of the Sparse DFT bandwidth estimation algorithm compared to a ground truth PSD, utilizing experimental ultrasound data from the PICMUS dataset.

the system to achieve highly accurate bandwidth estimates comparable to very large FFTs, while maintaining the minimal resource utilization benefits characteristic of this sparse architecture. A demonstration of this is presented in Figure 3.2.

3.1.3. Memory Footprint

The primary advantage of the proposed Sparse DFT architecture is its ability to process data in a true streaming fashion. Unlike a standard FFT, which necessitates buffering an entire frame or window of samples before computation can start, the Sparse DFT updates its spectral estimates incrementally as each sample arrives. By restricting the analysis to K specific frequency bins and eliminating input window buffering, this approach yields a significant reduction in memory footprint.

To quantify this advantage, we compare the theoretical lower bound of the memory footprint for both architectures. In an ideal I/Q-demodulated FFT implementation, the system must buffer the real and imaginary components of the full time-domain window. The required memory capacity M_{FFT} in bits is therefore given by:

$$M_{FFT} = 2 \cdot N \cdot B_{sample} \quad (3.14)$$

where N is the window length and B_{sample} is the bit-width of the input samples.

In contrast, the streaming Sparse DFT requires no input buffering. Memory is required solely for the state variables of the analysis: the complex accumulators and the phase

3. System and Algorithm Design

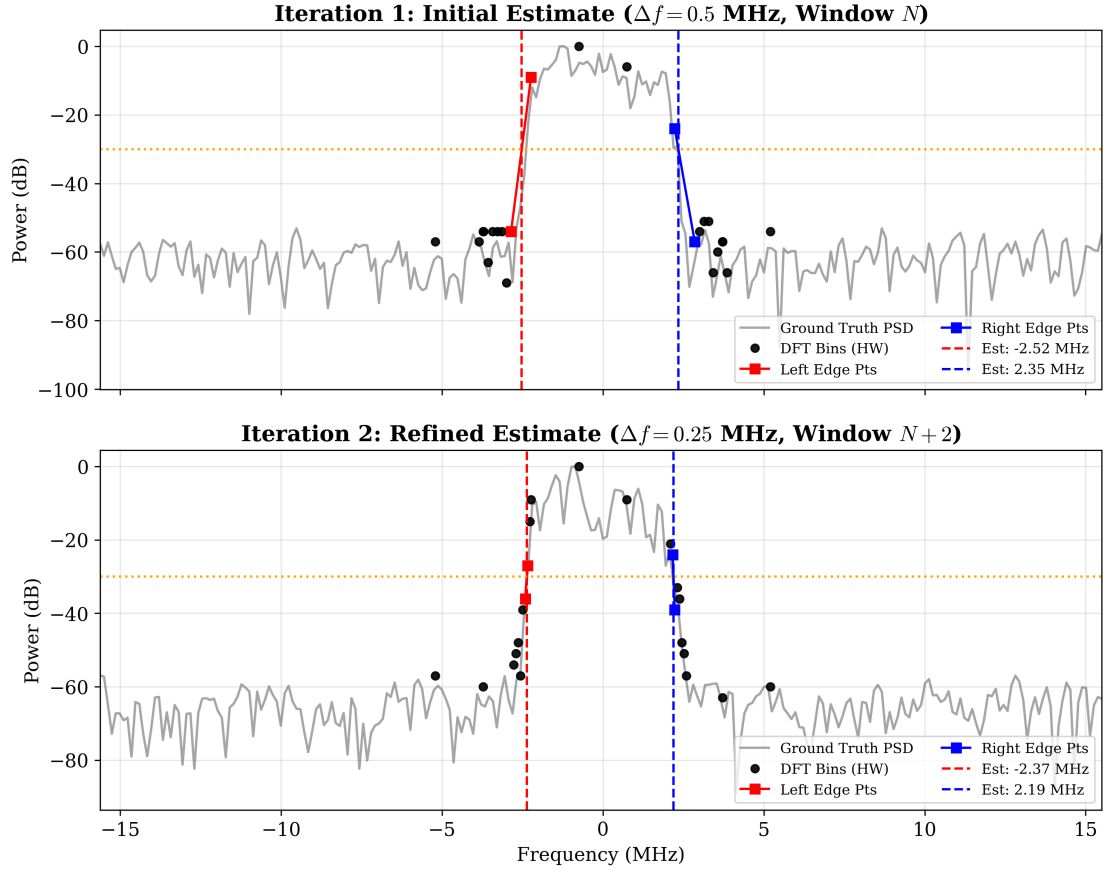


Figure 3.2.: Iterative Refinement Demonstration. Initial fine clusters are deliberately shifted away from the true bandwidth edges to highlight the update process. The subsequent iteration repositions the clusters to be centered around the estimated edges.

3. System and Algorithm Design

accumulators for the oscillator values. The memory footprint M_{sparse} is defined as:

$$M_{sparse} = K \cdot (2B_{acc} + B_{phase}) \quad (3.15)$$

where K is the number of calculated frequency bins ($K = |S|$), B_{acc} is the bit-width of the complex accumulator components, and B_{phase} is the bit-width of the phases for the complex oscillator values.

Table 3.1 presents a comparison of these models using the system’s specific parameters: 16-bit input samples ($B_{sample} = 16$), 36-bit accumulators ($B_{acc} = 36$), 16-bit phase accumulators ($B_{phase} = 16$), and a sparse set of $K = 24$ bins.

	FFT				Sparse DFT
Window Length	256	512	1024	2048	Configurable
Memory Footprint [Bits]	8192	16384	32768	65536	2112

Table 3.1.: Theoretically ideal memory footprint comparison between standard buffered FFTs of varying lengths and the proposed streaming Sparse DFT ($K = 24$).

While these figures represent an ideal theoretical minimum rather than precise FPGA resource utilization, they illustrate the scalability of the Sparse DFT. Crucially, as already mentioned in Section 3.1.1, the memory requirement for the Sparse DFT is decoupled from the window length N , significantly reducing the required memory. A detailed breakdown of the actual hardware resource utilization is provided in Chapter 5.

It is important to note that this memory efficiency comes at the cost of arithmetic complexity. As established in Section 2.4.2, the FFT algorithm offers a computational complexity of $\mathcal{O}(N \log N)$. The Sparse DFT, calculating K bins individually over N samples, has a complexity of $\mathcal{O}(K \cdot N)$. Since typical configurations satisfy $K > \log N$, the Sparse DFT requires a higher total number of arithmetic operations per frame. However, in a hardware context, this increased operational count does not strictly translate to increased resource usage. Because the Sparse DFT operates in a serialized, streaming fashion, arithmetic units are efficiently reused every clock cycle, keeping the physical logic area small despite the higher total operation count. The architectural choices are detailed further in Chapter 4.

3.2. Adaptive Decimation Filter

By leveraging the knowledge of the transducer’s precise bandwidth edges, we can determine the maximum decimation ratio that maintains signal integrity while minimizing data throughput. To prevent aliasing and provide sufficient oversampling, we apply a safety factor of 2 relative to the Nyquist rate, requiring the AFE sampling frequency to

3. System and Algorithm Design

be at least four times the highest frequency containing information ($f_s \geq 4f_{max}$). This yields an integer decimation ratio of:

$$M = \left\lfloor \frac{f_s}{4f_{max}} \right\rfloor \quad (3.16)$$

Within the AFE, this decimation is executed by a hardware block combining a configurable symmetrical FIR low-pass filter (LPF) and a decimator.

A critical challenge in this process is the preservation of signal components at the transducer's bandwidth edges. These components inherently suffer from significant physical attenuation (typically -6 dB). Standard decimation filters further suppress these frequencies through the roll-off of the LPF frequency response. Consequently, when the filtered signal is re-quantized to 16 bits, these attenuated components risk falling close to the quantization noise floor, resulting in a permanent loss of information.

To mitigate this, we propose an edge-boosting filter profile that acts as a prewhitening transform. As described by Thon et al. [18], this technique effectively flattens the spectral response by amplifying the attenuated frequency edges, which maximizes the utilization of the ADC's dynamic range and improves the Signal-to-Quantization-Noise Ratio (SQNR). While this equalization could theoretically be performed downstream on the FPGA, implementing it directly within the AFE's configurable decimation filter offers two distinct advantages:

1. **Resource Efficiency:** It leverages the existing AFE hardware, eliminating the need for a redundant FIR filter stage on the FPGA and thus conserving logic resources.
2. **Re-quantization Fidelity:** By boosting the signal before the AFE's final 16-bit truncation, we achieve a precision gain over a generic post-processing approach. Amplifying the signal prior to re-quantization preserves low-amplitude details that would otherwise be lost to quantization error.

To demonstrate the precision advantage of the proposed edge-boosting filter over a standard LPF, a comparative simulation was conducted using a synthetic sine wave centered at the bandwidth edge (3 MHz) of a hypothetical transducer.

Two filtering scenarios were evaluated using a fixed AFE LPF tap constraint of $16 \cdot M$, with the integer decimation ratio M set to 4:

- **Scenario A (Generic):** A standard symmetrical FIR LPF using a Hamming window.
- **Scenario B (Custom):** The proposed adaptive edge-boost filter, designed using the Arbitrary Magnitude Filter method in MATLAB [19].

The signal processing chain is shown in Figure 3.3 and the results are illustrated in Figure 3.4. Figure 3.4.a) depicts the initial analog input sine wave before digitization. Figure 3.4.b) compares the frequency response of both filters, highlighting the magnitude boost in Scenario B. The resulting digital signals, re-quantized to 16 bits after decimation,

3. System and Algorithm Design

are shown in Figure 3.4.c). Finally, Figure 3.4.d) visualizes the effective quantization error referred back to the filter input, demonstrating the reduced error magnitude in the boosted scenario.

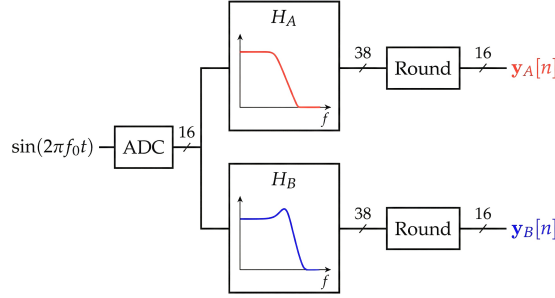


Figure 3.3.: Signal processing chain detailing the theoretical decimation filter performance comparison between a generic LPF (red) and the custom edge-boost filter (blue).

3.2.1. Signal-to-Noise-and-Distortion Ratio Comparison

To quantify the performance improvement, we calculate the Signal-to-Noise-and-Distortion Ratio (SINAD) for both scenarios, following the IEEE Standard 1241 [20]. The SINAD is defined as the ratio of the root-mean-square (RMS) signal amplitude to the RMS noise and distortion (NAD). The NAD is derived by fitting an ideal sine wave $x'[n]$ to the system output $x[n]$ and computing the residual error:

$$\text{NAD} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (x[n] - x'[n])^2}, \quad (3.17)$$

where N represents the total number of samples. The SINAD is then calculated as:

$$\text{SINAD} = \frac{A_{\text{rms}}}{\text{NAD}}, \quad (3.18)$$

with the RMS signal amplitude defined as:

$$A_{\text{rms}} = \frac{\text{Sine Wave Peak Amplitude}}{\sqrt{2}} \quad (3.19)$$

Applying these metrics yields a SINAD_A of 33.89 dB for the generic filter and a SINAD_B of 38.81 dB for the custom edge-boost filter. This demonstrates a net signal quality improvement of:

$$\Delta \text{SINAD} = \text{SINAD}_B - \text{SINAD}_A = +4.92 \text{ dB} \quad (3.20)$$

This increase in SINAD confirms that amplifying the signal at the bandwidth edges prior to re-quantization significantly preserves signal fidelity against quantization noise.

3. System and Algorithm Design

3.2.2. Signal-to-Quantization-Noise Ratio Comparison

While the IEEE SINAD metric confirms an overall improvement, it includes the input thermal noise, which is common to both scenarios. To isolate the specific benefit of the edge-boost filter, we analyze the re-quantization SQNR. This metric focuses exclusively on the noise introduced during the final truncation from high-precision internal values to the 16-bit output.

We define the re-quantization error $e[n]$ as the difference between the high-precision filter output before rounding, $y_{high}[n]$, and the final 16-bit integer output, $y_{int}[n]$. To make it a fair comparison, this error must be referred back to the input scale by normalizing it with the filter's gain G at the test frequency:

$$e_{ref}[n] = \frac{y_{high}[n] - y_{int}[n]}{G} \quad (3.21)$$

The SQNR is then calculated as the ratio of the signal power P_s to the power of this input-referred quantization error P_e

$$\text{SQNR}_{\text{dB}} = 10 \log_{10} \left(\frac{P_s}{P_e} \right) = 10 \log_{10} \left(\frac{\frac{1}{N} \sum_{n=0}^{N-1} (x[n])^2}{\frac{1}{N} \sum_{n=0}^{N-1} (e_{ref}[n])^2} \right) \quad (3.22)$$

By applying this method, we obtain $\text{SQNR}_A = 85.22$ dB for the generic filter and $\text{SQNR}_B = 97.11$ dB for the custom edge-boost filter. This yields a substantial improvement of:

$$\Delta \text{SQNR} = 97.11 \text{ dB} - 85.22 \text{ dB} = +11.88 \text{ dB} \quad (3.23)$$

This 11.88 dB gain represents the specific reduction in the re-quantization noise floor relative to the signal level, which is achieved solely by utilizing the available dynamic range more effectively. We can translate this dB improvement directly into an **Effective Resolution Improvement** in bits.

Note that we are specifically not using the metric **Effective Number of Bits** (ENOB), which is defined by the IEEE Standard 1241 as follows:

$$\text{ENOB} = \log_2 \left(\frac{\text{FSR}/G}{\text{NAD}\sqrt{12}} \right) = \log_2(\text{SINAD}) - \log_2 \left(\frac{2G \times A}{\text{FSR}} \right) - \frac{1}{2} \log_2(1.5) \quad (3.24)$$

FSR describes the full-scale range of the ADC, G is the measured gain and A is the amplitude of the fitted sine wave during the SINAD calculation as described in the equations (3.17)-(3.19).

The ENOB metric is ill-suited for quantifying the benefits of equalization in this application because it penalizes the gain G introduced by the boost filter, interpreting it as a reduction in system headroom (FSR/G). In other words, the IEEE definition of

3. System and Algorithm Design

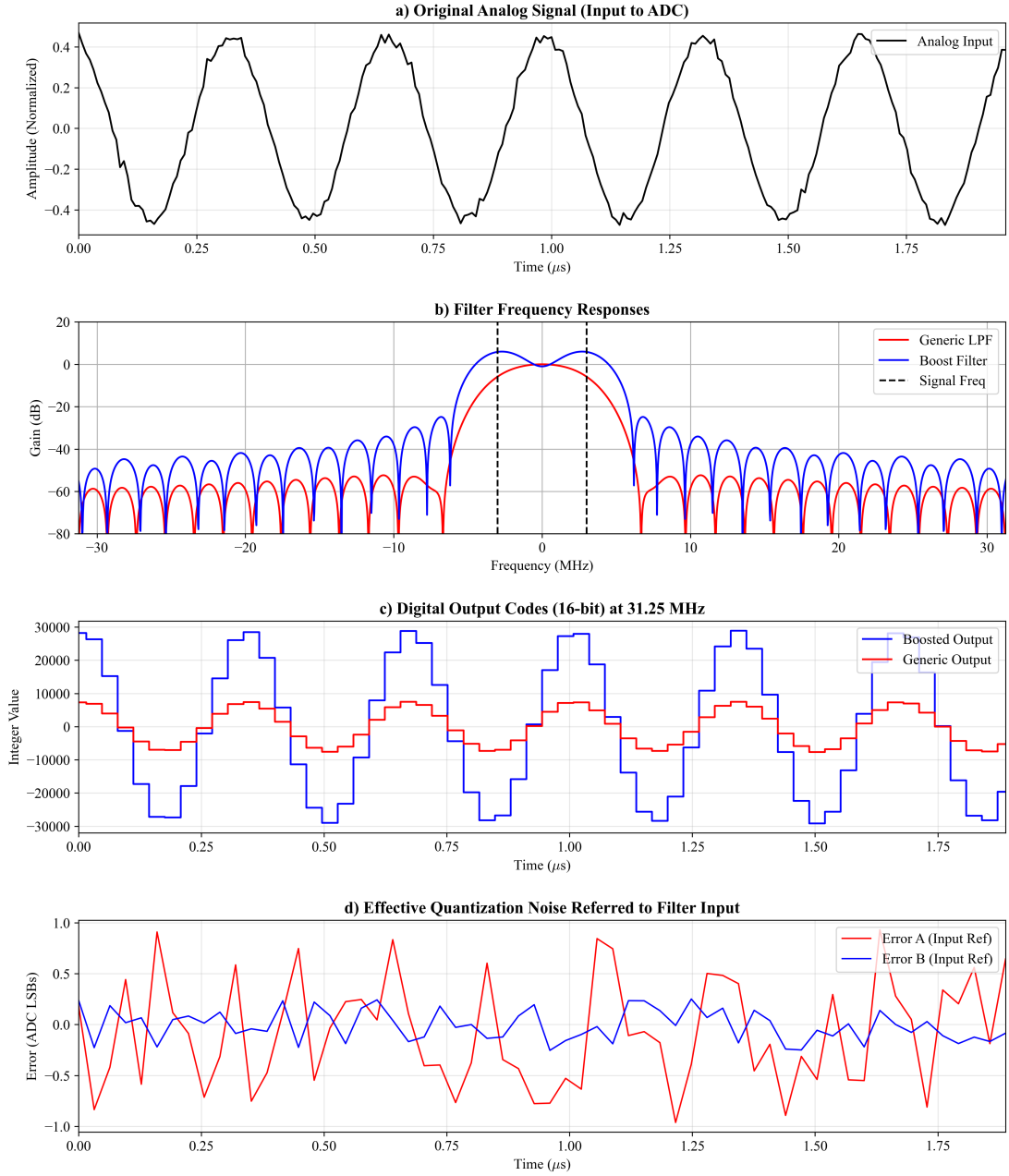


Figure 3.4.: Comparative analysis of decimation filter performance. a) The original analog input signal. b) Frequency magnitude response of the generic Hamming window LPF versus the custom edge-boost filter. c) The resulting 16-bit re-quantized digital output codes. d) The effective quantization noise error, referred to the filter input in ADC LSBs, illustrating the reduction in error magnitude achieved by the edge-boost architecture.

3. System and Algorithm Design

the ENOB metric prioritizes input dynamic range, respectively being able to accept a full-scale input, over signal fidelity.

However, in this specific use case, the signal of interest is known to be attenuated by the channel (approximately -6 dB at the cutoff frequency). Consequently, the upper dynamic range is unutilized, so the proposed boost filter intentionally sacrifices this available headroom to improve the SQNR during the re-quantization stage.

Therefore, instead of the standard Input-Referred ENOB, we define the **Effective Re-quantization Resolution** N_{eff} . This metric represents the actual bit-depth utilized by the signal relative to the fixed quantization noise floor. For a signal with amplitude A quantized by a system with a full-scale range FSR and a total number of bits N_{tot} , the effective resolution is reduced by the unused headroom:

$$N_{eff} = N_{tot} - \log_2 \left(\frac{FSR}{2A} \right) \quad (3.25)$$

Using the relationship between signal resolution and quantization noise derived in Equation (A.12) (see Appendix A.2), the SQNR can be approximated as:

$$SQNR_{dB} \approx 6.02 \cdot N_{eff} + 1.76 \quad (3.26)$$

We can now quantify the performance gain by comparing the effective resolution of the boosted system (Scenario B) against the generic system (Scenario A). The improvement, denoted as ΔN_{eff} , is derived by subtracting the SQNR of the two scenarios:

$$\begin{aligned} \Delta SQNR &= SQNR_B - SQNR_A \\ &\approx (6.02 \cdot N_{eff,B} + 1.76) - (6.02 \cdot N_{eff,A} + 1.76) \\ &= 6.02 (N_{eff,B} - N_{eff,A}) \\ &= 6.02 \cdot \Delta N_{eff} \end{aligned} \quad (3.27)$$

Solving for ΔN_{eff} yields the final metric:

$$\Delta N_{eff} = \frac{SQNR_B - SQNR_A}{6.02} = +1.97 \text{ bits} \quad (3.28)$$

This metric explicitly tells us how many additional bits of precision the generic system in Scenario A would need to achieve the same re-quantization SQNR performance as the edge-boost filter in Scenario B. Clearly, Scenario B shows an improvement over Scenario A.

3. System and Algorithm Design

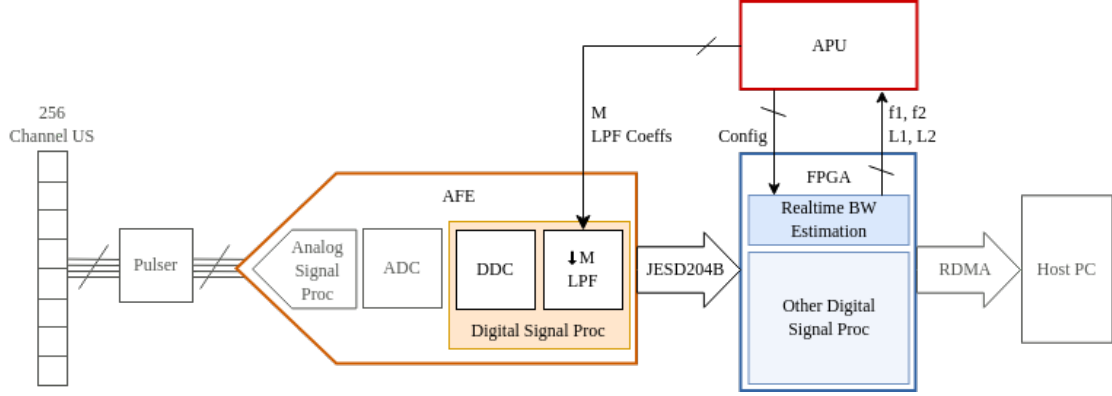


Figure 3.5.: Hardware Partitioning for Closed-Loop Control: FPGA for Real-Time Bandwidth Estimation, APU for Linear Interpolation and Control, and AFE for Low-Pass Filtering and Decimation

3.3. Hardware Architecture Overview and Partitioning

The overall system architecture and data flow for the proposed bandwidth estimation and adaptive compression scheme are illustrated in Figure 3.5. The processing pipeline is partitioned across three main stages: the AFE, the Programmable Logic (FPGA), and the Application Processing Unit (APU).

As established in Section 2.2.3, the AFE acts as the primary signal conditioning stage. It performs analog-to-digital conversion, digital I/Q demodulation, and the initial decimation. The resulting decimated I/Q data stream is transmitted to the FPGA via a high-speed JESD204B link.

The FPGA is dedicated to high-throughput, real-time signal processing. Upon reception, the serial JESD204B I/Q data is converted into a parallel AXI-Stream format. This stream feeds directly into the Streaming Sparse DFT block.

The FPGA is responsible for the computationally intensive, repetitive tasks that must operate at rates exceeding the sampling frequency. This includes:

1. Sparse DFT Calculation: Real-time phase accumulation and complex oscillator value calculation using CORDIC blocks for the set of frequency bins of interest.
2. Power Estimation: Conversion of complex accumulator values to logarithmic power and estimating the power threshold.
3. Edge Search: Determining the frequency bins (f_1, f_2) in between which the threshold crossing happens and their corresponding power levels (L_1, L_2) for both the left and right bandwidth edges.

Once the frame is processed, the FPGA sends these intermediate edge detection results to the APU.

3. System and Algorithm Design

The APU performs the system control, configuration, and post-processing analysis. The decision to partition the algorithm here is driven by the nature of the remaining mathematical operations. Specifically, the final calculation of the exact edge frequency f^* requires linear interpolation in the log-domain:

$$f^* = f_1 + (f_2 - f_1) \cdot \frac{P_{th} - L_1}{L_2 - L_1} \quad (3.29)$$

This operation includes a division, which is resource-expensive to implement in hardware logic but trivial for the APU. Furthermore, these calculations occur only once per frame, making the APU the ideal candidate for this task.

In addition to the linear interpolation, the APU acts as an adaptive controller. Before a frame begins, it calculates the necessary configuration parameters, specifically the frequency steps $\Delta\theta_k$ and window coefficients, based on the results of the previous iteration, as described in Section 3.1.1. These parameters are sent to the FPGA to configure the Sparse DFT block for the next acquisition cycle. Finally, based on the computed bandwidth edges, the APU determines the maximum permissible decimation factor M , adjusts the decimation filter strategy as detailed in Section 3.2 and changes the AFE configuration accordingly. By utilizing the internal AFE decimation filter, we leverage the pre-requantization precision benefits described previously and optimize the data link for the subsequent acquisitions.

The decimated and filtered I/Q data is fed to the host PC for further downstream processing and/or storage.

As illustrated in Figure 3.5, this architecture forms an independent closed-loop control system for the incoming I/Q demodulated data that always ensures that the system is meeting the bandwidth constraints between the FPGA and host PC in a weakly lossy manner.

Implementation

4.1. RTL Architecture

4.1.1. Top-Level Integration

The top module provides a top-level integration module of the FPGA design, combining the configuration, estimation, and analysis stages into a unified pipeline. It abstracts the internal complexity of the synchronization logic, exposing a simplified interface consisting of serial configuration ports, streaming I/Q inputs, and the final bandwidth edge indices (f_1 , f_2) and power values (L_1 , L_2).

A fundamental architectural decision was the selection of the system clock frequency relative to the input data rate to enable resource sharing. The upstream Analog Front End (AFE) digitizes signals at a rate of 125 Mega-Samples-Per-Second (MSPS). This high frame rate digital RF data is down-converted and sent to the FPGA for processing.

A baseline decimation factor of 4 is necessary due to the physical bandwidth constraints of the JESD204B interface. The system aggregates data from 8 ADC dies, effectively creating 16 distinct 16-bit streams (I and Q) per JESD lane. At the native sample rate of 125 MSPS, the aggregate throughput including 8b/10b encoding overhead inside the JESD interface would reach $16 \times 16 \times 125 \times 10/8 = 40$ Gbps. Since the transceiver lanes operate at a fixed rate of 10 Gbps, which corresponds to the 80X PLL mode relative to the 125 MHz clock, a decimation factor of 4 is required to reduce the aggregate data rate to match the physical lane capacity. This means the effective base-level sample rate arriving at the FPGA is:

$$f_{s,eff} = \frac{125 \text{ MSPS}}{4} = 31.25 \text{ MSPS} \quad (4.1)$$

4. Implementation

The FPGA logic is synthesized to operate at a clock frequency (f_{sys}) of 375 MHz. The ratio between f_{sys} and $f_{s,eff}$ defines the available processing budget in clock cycles per sample:

$$\text{Cycles per Sample} = \frac{f_{sys}}{f_{s,eff}} = \frac{375 \text{ MHz}}{31.25 \text{ MHz}} = 12 \text{ cycles} \quad (4.2)$$

This 12-cycle window per sample allows the sparse DFT accumulation architecture to be time-multiplexed as will be detailed in Section (4.1.3).

The internal data flow is orchestrated by the valid-ready handshakes of the sub-modules:

1. Configuration Phase

To minimize physical I/O usage, the design utilizes a serial configuration strategy. The `config_loader` deserializes the phase steps and bin indices via a narrow AXI-Stream interface. Once the `config_valid` signal is asserted, the system transitions to the active state.

2. Active Processing Phase

When the window of interest of the incoming I/Q stream arrives at the input of the FPGA, an enable signal is asserted and the `spectral_power_estimator` begins consuming the streaming I/Q samples. The module manages the domain crossing from the effective base-level sample rate (31.25 MHz) to the physical clock rate (375 MHz) via the sample holding mechanism detailed in Section 4.1.3.

3. Analysis Phase

Upon the completion of a window (e.g., 256 samples), the `dft_valid` signal triggers the `bandwidth_edge_detector`. Since the edge detection logic operates on the static power snapshot after the window is closed, it does not impose backpressure on the real-time processing. The final outputs (`f1`, `f2`, `L1`, `L2`) are updated once per frame, providing a synchronous, low-bandwidth measurement of the signal's spectral boundaries to the APU.

4.1.2. Configuration and Control Interfaces

The control interface serves as the bridge between the external host system and the real-time processing pipeline. It comprises two distinct functional blocks: A runtime configuration loader that updates spectral parameters via AXI-Stream, and a precision timing controller that synchronizes the pipelined oscillators and the DFT accumulation with the input data stream.

4. Implementation

Runtime Configuration Loading

The `config_loader` module functions as a serial-to-parallel interface that enables the host to update the DFT bin frequencies of interest without requiring FPGA reconfiguration. It corresponds to a standard AXI4-Stream slave interface that operates using a finite state machine (FSM) that strictly enforces the following two-stage loading protocol:

1. **Frequency Steps Loading:** The first N transfers, where N corresponds to `NUM_BINS`, are interpreted as the phase increments (`freq_steps`) for the phase accumulators that determine the input for the CORDIC oscillators as described in Equation (3.5).
2. **Bin Indices Loading:** The subsequent N transfers carry the frequency bin indices (`freq_bin`) used for the bandwidth edge finding as described in Equation (3.11).

The `config_loader` writes this serially accepted data into a set of parallel registers and asserts a `config_valid` signal once $2N$ transfers are successfully latched. Following the standard AXI-Stream handshake mechanism, the loader supports discontinuous inputs, allowing the host to insert gaps between data words as long as the valid signal is managed correctly. This architecture allows the spectral power estimator to switch between different frequency sub-bands (e.g., zooming into a specific target range) in real-time between two subsequent frames.

Pipeline Synchronization

The `counter_controller` module orchestrates the timing of the spectral estimation window. Its primary design challenge is managing the latency mismatch between the deep CORDIC pipeline and the input data stream. Since the CORDIC-based oscillators require a fixed startup latency of 20 cycles to produce valid phase outputs in our current configuration, they must be enabled prior to the arrival of valid sample data.

The controller implements a parametrizable counter that tracks three distinct delay phases:

1. **Starting Up the Oscillator:** The controller asserts `osc_enable` and `osc_phase_tvalid` in advance, based on the `CORDIC_DELAY` parameter, ensuring the sinusoidal outputs stabilize exactly when the processing window begins.
2. **Processing the Window:** Once the window of interest inside a full I/Q frame arrives, the controller asserts a `start` and `sample_valid` signal to gate the accumulation logic for the duration of the frame.
3. **Terminating the Window:** A `last_sample` flag is generated at the exact end of the window that signalizes the end of the DFT accumulation for the window of interest.

4. Implementation

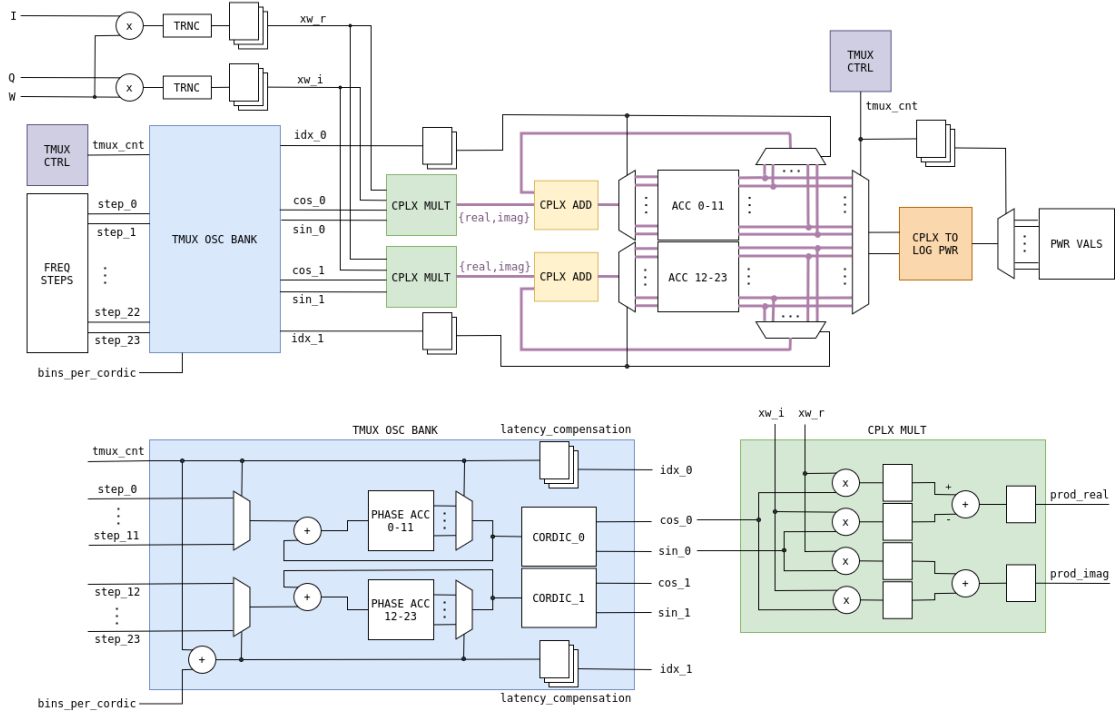


Figure 4.1.: Spectral Power Estimator Architecture: Time-multiplexed CORDIC blocks generate oscillator values for the subsequent DFT accumulation. The logarithmic power conversion is fully serialized.

This precise, counter-based synchronization decouples the complex pipeline latency from the external system, allowing the top-level integration to operate with a simple "Window Start"-index trigger while the controller handles the internal cycle-accurate sequencing.

A critical constraint of the current counter_controller implementation is the requirement for **burst-mode sample data transmission**. Specifically, the input I/Q stream must maintain continuous validity for the duration of the processing window. This constraint arises from the fact that the control scheme of the CORDIC oscillators is decoupled and predictive. Specifically, the CORDIC oscillators are triggered prior to sample arrival to compensate for their pipeline depth. As detailed in Section 4.1.1, the system is designed to run at 375 MHz. Given the decimated input sample rate of 31.25 MHz (125 MSPS / 4) and a CORDIC latency of 20 cycles at 375 MHz, full compliance with sparse AXI-Stream protocols (i.e., discontinuous inputs) would require an input elasticity buffer. A shallow FIFO with a depth of at least two I/Q samples would be sufficient to bridge the oscillator latency gap, allowing the oscillator enable signal to be dynamically gated by the input validity signal without a loss of phase synchronization.

4. Implementation

4.1.3. Spectral Power Estimation

The core of the signal processing pipeline is the Spectral Power Estimation block. As described in Section 3.1.1, this implementation utilizes a sparse DFT approach unlike FFT architectures that calculate the full spectrum N points at a time. This architecture was selected because the application requires high-resolution monitoring of only a specific subset of frequency bins rather than the entire bandwidth.

To maximize resource efficiency on the FPGA, the module utilizes a time-multiplexed architecture. As already established in Section 4.1.1, the system has a processing budget of 12 clock cycles per input sample. The design exploits these idle cycles to serialize the arithmetic operations to calculate multiple frequency bins sequentially inside the time of one single I/Q sample.

Time-Multiplexed Oscillator Bank

The oscillator values required for the DFT computation are generated in real-time using CORDIC logic instead of storing pre-calculated sine/cosine tables in Block RAM (BRAM), which would require significant memory for high-precision phase steps and would need to be recomputed and reloaded for every frequency bin adjustment in between frames.

To support 24 frequency bins within the 12-cycle processing window, the bins are split into two parallel CORDIC pipelines, each handling 12 bins time-multiplexed:

- CORDIC 0: Generates phasors for bins 0-11.
- CORDIC 1: Generates phasors for bins 12-23.

The module maintains an array of 24 independent phase accumulators (`pase_acc_q`), which are updated in an interleaved fashion. A counter iterates from 0 to 11 and on each count i , the logic updates the phase for bin i and $i + 12$ simultaneously by adding their respective `freq_steps`.

Since the Xilinx CORDIC IP cores have a deep pipeline of 20 cycles, this module must also ensure that the generated sine/cosine values align with the correct bin index at the output. For this purpose, the module implements a bin index shift register (`counter_pipe`) that delays the current phase input index counter value by exactly 20 cycles. When valid data arrives at the output of the CORDIC, the shift register outputs the corresponding bin index which strictly ensures phase coherency for the DFT accumulation logic in the next stage.

4. Implementation

Streaming Sparse DFT Core

The `dft_accumulation_cordic_tmux_streaming` module performs windowing, DFT computation of the incoming I/Q samples with the generated local oscillators and accumulation across the processing window. To meet the real-time throughput requirement of calculating 24 frequency bins within the 12 available clock cycles per I/Q sample, the module must process two frequency bins per clock cycle.

To achieve timing closure at the target frequency of 375 MHz, the datapath is deeply pipelined into four distinct stages. This organization separates the heavy arithmetic operations into isolated clock cycles, reducing the critical path significantly. The pipeline stages are defined as follows:

- **Stage 1: Windowing & Sample Holding**

This stage accepts the decimated I/Q samples and multiplies them by the corresponding real-valued Hanning window coefficient (`window_coeff_i`). The resulting products are immediately truncated to 20 bits (`STAGE1_WIDTH`) to manage bit growth.

A feature of this stage is its sample holding mechanism. Since the time-multiplexed architecture requires the same input sample to be multiplied to 12 oscillator values for different bins sequentially, the windowed result must remain at the input for the following stage for 12 clock cycles. Essentially, the windowing is carried out once and the result is frozen while the oscillator bank output indices cycle through bins 0-11 and 12-23.

- **Stage 2a & 2b: Pipelined Complex Multiplication**

The multiplication $(xw_r + j * xw_i) * (W_r + j * W_i)$ of the windowed I/Q sample and the oscillator value represents a significant combinational load. The result $(xw_r * W_r - xw_i * W_i) + j * (xw_r * W_i + xw_i * W_r)$ requires four parallel multiplications and a subsequent addition and subtraction. With bit widths of 20 bits for `xw_r` and `xw_i` and 16 bits for `W_r` and `W_i`, the initial products that need to be added/subtracted have a width of 36 bits. Performing both the multiplications and addition/subtraction in the same clock cycle would give us a very long critical path that ends up violating the timing constraints. Because of this, the complex multiplication itself is pipelined into two stages. Stage 2a calculates the four partial products. Since CORDIC outputs for two distinct bins are generated in parallel, this stage infers eight DSP48 units.

- **Stage 3: Accumulation**

This is the final stage of the sparse DFT accumulation. It routes and adds the calculated products to the correct accumulator registers. This stage relies on the signals `prod_bin_0_q` and `prod_bin_1_q`. These are pipelined versions of the oscillator bin indices to address the distributed registers `A_real_q` and `A_imag_q` for the 24 distinct frequency bins.

4. Implementation

During timing analysis, two main things were observed:

1. Without pipelining the bin indices (`prod_bin_0/1`), the critical path extended across the entire accumulation update cycle. This path included decoding the bin index, retrieving the current accumulator value via a wide multiplexer, performing the addition of the new product, and demultiplexing the result back to the target register. This deep Read-Modify-Write logic structure resulted in severe negative slack. Consequently, the indices (`prod_bin_0/1_q`) were pipelined to decouple the address decoding from the arithmetic accumulation, splitting the operation across distinct clock cycles.
2. Even with this pipelining, the timing constraints were initially not met due to excessive net delays. The bin index signals effectively function as write-enables for the entire array of distributed accumulators, resulting in a very high fanout. This high fanout forced the placement and routing tool to utilize long interconnects, creating setup violations at 375 MHz. To resolve this, a `MAX_FANOUT` synthesis attribute was applied directly in the RTL:

```
(* MAX_FANOUT = 12 *) logic [COUNTER_WIDTH-1:0] prod_bin_0_q, prod_bin_0_d;  
(* MAX_FANOUT = 12 *) logic [COUNTER_WIDTH-1:0] prod_bin_1_q, prod_bin_1_d;
```

By capping the fanout at 12, the synthesis tool is forced to replicate the registers driving these signals, if they have a higher fanout. This physically localizes the control logic to the specific accumulator groups it drives, significantly reducing net delay and allowing the design to meet the 375 MHz timing requirement. This effectively got rid of all the remaining timing violations at a minimal cost, since these bin indices are only 5 bits wide.

The entire sparse DFT accumulation process is governed by a three-state FSM (IDLE, ACCUMULATE, DONE). To ensure the "last sample" flag correctly triggers the end of the frame after traversing the pipeline, the `last_sample` signal is propagated through a pipeline of the same length. This guarantees that the FSM transitions to DONE only after the last bin of the final sample has been fully accumulated.

Logarithmic Power Conversion

Following the accumulation phase, the 24 complex DFT results must be converted into a scalar logarithmic power representation in dB for the bandwidth estimation. Unlike the DFT core, which is constrained by the real-time arrival of input samples (requiring 24 bin updates every 12 cycles), the power conversion is triggered after the DFT has been fully computed and does not rely on any incoming I/Q samples anymore. This decoupled timing budget allows for a fully time-multiplexed architecture, where a single hardware converter instance processes all 24 bins sequentially, minimizing resource utilization.

The `complex_to_log_power_tmux` module manages this serialization. It implements an FSM that iterates through the `A_real` and `A_imag` accumulator registers, feeding one

4. Implementation

complex pair per clock cycle into the conversion pipeline. A counter tracks the pipeline depth ($L = 5$) to align the resulting power values with the correct output registers, ensuring that the converted values are written back to the correct bin indices.

To avoid the cost of hardware floating-point logarithms or large look-up tables, the `complex_to_log_power` module, which is responsible for the actual power calculation, implements a precise fixed-point friendly approximation:

$$\begin{aligned} P_{dB} &= 10 \cdot \log_{10} (I^2 + Q^2) = 10 \cdot \frac{\log_2 (I^2 + Q^2)}{\log_2(10)} \\ &\approx \frac{10}{3.32} \cdot \log_2 (I^2 + Q^2) \approx 3 \cdot \log_2 (I^2 + Q^2) \end{aligned} \quad (4.3)$$

The actual implementation simplifies this term even further. It only calculates the integer $\log_2(\cdot)$, which essentially corresponds to finding the most significant bit (MSB).

$$P_{dB} \approx 3 \cdot \left\lfloor \log_2 (I^2 + Q^2) \right\rfloor \quad (4.4)$$

This computation is broken down into a 4-stage pipeline:

- **Stage 1a: Squaring**
The signed complex input accumulator bins, which were truncated to 18-bits each, are squared using two DSP48 slices. This operation is isolated in its own pipeline stage because the 18x18-bit multiplication is a resource-intensive operation.
- **Stage 1b: Magnitude Summation**
The squared results are registered and then summed ($I_2 + Q^2$). Performing the addition and the multiplication in different clock cycles splits the critical path.
- **Stage 2: Integer Logarithm**
In this stage, the module calculates $\lfloor \log_2(P) \rfloor$ using a priority encoder. This logic scans the sum from the MSB down to the LSB to find the position of the leading '1'. This bit index represents the integer power of 2.
- **Stage 3: Scaling**
The integer index is multiplied by 3 to achieve the expression in Equation (4.4). This is implemented efficiently using shift-and-add logic: $3 * x = (x \ll 1) + x$. The result is registered as the final 8-bit power value.

This pipelined approach provides a latency of five cycles per bin, four from the power calculation itself inside `complex_to_log_power` and one from the registered time multiplexing index in `complex_to_log_power_tmux`. This pipeline allows the converter to run at the full system clock frequency, completing the entire 24 bin conversion in just 29 clock cycles (24 bins + 5 cycles latency) after the window of interest ends. This corresponds to less than 3 I/Q samples.

4. Implementation

4.1.4. Bandwidth Edge Detection

Once the spectral power estimation is complete, the 24 bin power values are analyzed to determine the bandwidth. This process is divided into the following sequential phases: Determining the dynamic power threshold based on the spectral peak, and identifying the frequency bin indices where the signal power crosses this threshold.

Peak Search and Thresholding

To establish a robust noise-relative bandwidth metric, the system uses a dynamic thresholding approach (e.g., $P_{max} - 30\text{dB}$). This requires identifying the global maximum power across all frequency bins.

Searching for a maximum value in a parallel array is traditionally an iterative process. However, to maintain high throughput, the `find_max_power` module implements a pipelined binary reduction tree. Instead of comparing bins sequentially ($\mathcal{O}(N)$), the module compares pairs of neighboring bins in parallel stages. The maximum of each stage are forwarded to the next, halving the number of candidate bins in each clock cycle. This architecture ensures a deterministic latency of $\lceil \log_2(N) \rceil + 1$ cycles (6 cycles for 24 bins), allowing the downstream logic to trigger exactly when the maximum is valid.

The `calc_abs_threshold` module accepts this global maximum and subtracts the configured threshold parameter (`THRESHOLD_DROP`). It includes an underflow protection logic: If the peak signal strength is weaker than the threshold drop (indicating a signal buried in noise), the absolute threshold is clamped to zero and a flag `threshold_ok_o` is set to zero, indicating that the bandwidth analysis fails for the given window. The entire module is fully synchronous and adds a single clock cycle of latency.

Bandwidth Edge Crossing Logic

The edge detection logic locates the specific frequency bins where the spectral power transitions from "signal content" to only "noise content". This is performed by two independent modules `find_bw_left_edge_absolute` and `find_bw_right_edge_absolute`, which scan the spectrum in opposite directions starting from the center frequency bin.

Unlike the parallel max search, edge detection is implemented as a serial linear scan using an FSM. This approach was selected because the specific index of the crossing is required, not just the existence of one.

- **Left Edge Search**

The FSM initializes at the center bin index ($N/2 - 1$) and iterates downwards toward index 0 (negative frequencies). It monitors a falling edge condition where the inner bin L_{in} is above the threshold and the outer neighbor L_{out} is below it, as described in Equation (3.11).

4. Implementation

- **Right Edge Search**

Completely analogously, the right edge detector scans from $N/2$ upwards to $N - 1$, looking for the corresponding transition on the positive side.

The FSM evaluates one bin per clock cycle for both modules each. Since the maximum scan depth is $N/2$ (12 bins), the edge detection completes rapidly within the time of one I/Q sample. There is no early exiting, meaning that the logic inherently prioritizes the outermost edges, ensuring that spectral dips within the main signal lobe do not prematurely terminate the bandwidth measurement.

4.2. Verification Methodology

4.2.1. Simulation Framework

To bridge the gap between high-level mathematical models and low-level hardware implementation, the verification strategy uses a reference model written in Python in combination with a SystemVerilog hardware simulation and a Python-based result analysis. This workflow leverages Python's extensive signal processing libraries for the stimuli generation and result analysis, while utilizing SystemVerilog for cycle-accurate timing verification of the Device Under Test (DUT). Stimuli generation files and testbenches are provided for all the submodules. The behavioral simulation flow for the top module is automated via a master Bash script (`full_simulation_flow.sh`), which orchestrates the verification process across three distinct steps:

1. **Step 1: Stimuli and Reference Generation**

The verification cycle begins with the execution of the `generate_vectors_top.py` script which, as the name suggests, generates the test vectors. It reads the raw PICMUS RF ultrasound datasets (e.g., experimental `contrast_speckle` or in vivo `carotid_cross` data) and processes them through the Virtual AFE pipeline (detailed in Section 4.2.2) to produce hardware accurate input vectors corresponding to the specification of the AFE that is used in this system. Then, the golden model (`complete_system_model`) is applied onto this generated data to produce the expected output values, consisting of the bandwidth edge indices and power levels for every test case. Both the generated I/Q samples and the golden model results are converted to fixed-point precision and written to a `.txt` file which is used to drive the DUT inside the testbench and for post-simulation comparison, as described in Section 4.2.4.

2. **Step 2: Cycle-Accurate Hardware Simulation**

The actual hardware verification is performed using the Xilinx Vivado Simulator (`xsim`). The SystemVerilog testbench is designed to be a generic file-reader instead of generating random stimuli internally. It parses the vector files generated in Step 1 and drives the DUT inputs cycle-by-cycle, respecting the AXI-Stream flow

4. Implementation

control and pipeline latency constraints. As the DUT processes the frames, the testbench monitors the outputs and logs the resulting bandwidth estimates and power values into CSV files for further analysis in Step 3.

To maximize simulation throughput, the automation script separates the compilation and execution stages. The design is compiled and elaborated once using a Tcl build script (`compile_elaborate.tcl`). Subsequently, the simulation executable is repeatedly invoked for each dataset using the `run_sim.tcl` script, enabling a single compilation to be reused across multiple simulation runs and thereby significantly reducing overhead when batch-processing thousands of test cases.

3. Step 3: Analysis and Reporting

Upon completion of the simulation, the resulting CSV files can be parsed by the `analyze_results.py` script that compares the DUT outputs against ground truth references. It is important to mention that in this stage, the results are not compared to the expected results of the hardware-accurate golden model, this is already verified inside the testbench itself. In this step, the actual ground truth bandwidth edges are computed using a full FFT and are then compared to the outputs of the hardware module. This catches both inaccuracies that are caused by the hardware modules and algorithmic limitations itself.

The simulation script is designed for robust batch processing and is capable of iterating through multiple large datasets without manual intervention. This ensures that the hardware is validated against a diverse set of physiological signals.

4.2.2. Stimuli Generation

As previously mentioned, the simulation framework utilizes the PICMUS dataset to evaluate the hardware design under realistic conditions. This benchmark dataset provides raw RF and I/Q demodulated data acquired from simulation, phantom experiments and in vivo carotid scans, serving as the ground truth for the verification process. Since the FPGA expects a decimated baseband I/Q stream at 31.25 MSPS, the raw RF data from the dataset is processed via a Virtual AFE model that is implemented in Python.

1. The script `virtual_afe.py` upsamples the PICMUS RF data, which was acquired at a sampling frequency of 20.832 MHz, to the target ADC clock frequency of 125 MHz using a polyphase resampling filter (`scipy.signal.resample_poly`).
2. Before digital down-conversion, the high-rate RF data undergoes band-pass filtering centered at the transducer modulation frequency (5.21 MHz). The signal is then demodulated via quadrature mixing and low-pass filtered to remove the high-frequency components exactly as described in Section 2.2.2 and A.1. To ensure strict spectral purity for verification, these filters are implemented as ideal brick-wall filters in the frequency domain using FFT/IFFT operations.

4. Implementation

3. Finally, the 125 MSPS I/Q stream is decimated by a baseline factor of 4 using an anti-aliasing FIR filter, resulting in the target 31.25 MSPS data rate required by the hardware interface. The decimation rate is a configurable parameter of the Virtual AFE and enables the implementation of higher decimation factors determined by the bandwidth estimation described in Chapter 3.

A critical function of the stimuli generator is to map the floating-point data to the fixed-point dynamic range of the hardware. The `generate_vectors_top.py` script applies a frame-wise dynamic scaling according to

$$S_{fixed} = S_{float} \cdot \frac{2^{N-1} - 1}{\max(|S_{frame}|)}, \quad (4.5)$$

where $N = 16$ denotes the input bit width. This scaling adapts the arbitrarily scaled floating-point I/Q samples to the available fixed-point range of the hardware representation, which ensures a consistent numerical mapping while avoiding arithmetic overflow.

4.2.3. Validation of the Virtual AFE

To validate that the Virtual AFE accurately reproduces the characteristics of the original signal, a rigorous randomized validation was conducted against the ground-truth I/Q data provided in the PICMUS dataset. The objective was to verify that the upsampling, filtering, and demodulation steps introduce negligible error such that the synthetic I/Q data is proven to be realistic and can be used for simulation and verification.

For each target dataset, the framework generates 501 unique test cases consisting of 50 randomly selected transducer channels, 10 randomly selected transmission angles per channel and 1 fixed control case (Channel 64, Angle 37) for consistent plotting. This evaluation was performed across the following four datasets:

- experiments/contrast_speckle
- experiments/resolution_distorsion
- in_vivo/carotid_cross
- in_vivo/carotid_long

A direct element-wise comparison between the generated I/Q stream and the pre-demodulated ground-truth PICMUS data is not possible, since the time grids do not align due to the difference in sampling frequency. To resolve this, the validation script performs linear interpolation of the generated signal onto the specific time axis of the reference data before calculating the error. Following this alignment, the following metrics were taken for every dataset:

4. Implementation

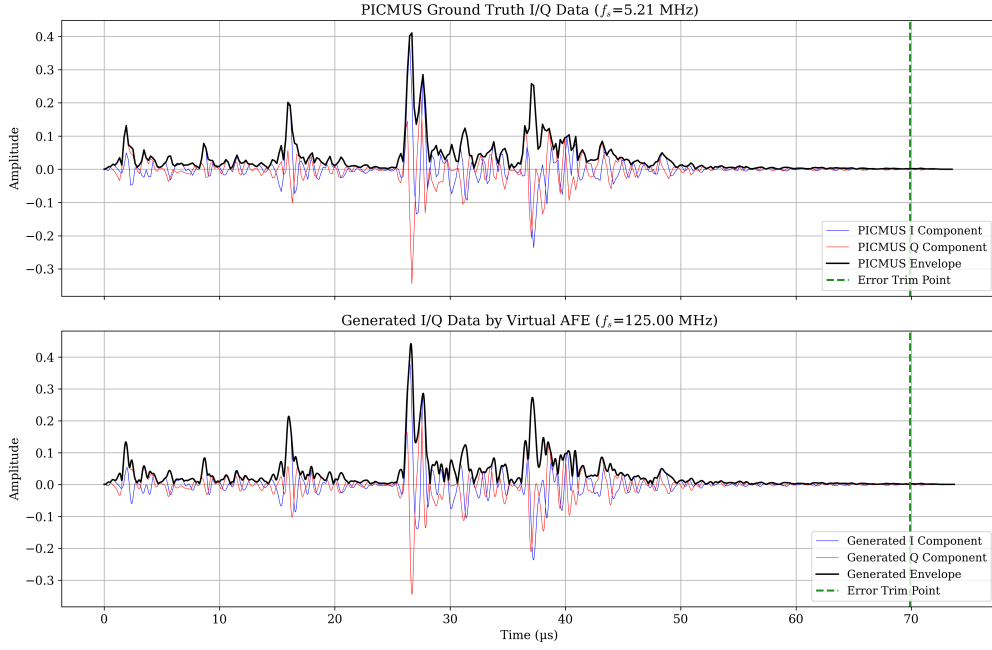


Figure 4.2.: Comparison of a ground-truth PICMUS I/Q frame with upsampled data generated by the Virtual AFE. The depicted frame corresponds to channel 64, angle 37 of the in vivo carotid cross dataset.

- i. Average Maximum Error: The peak absolute deviation between the interpolated generated signal and the reference signal, averaged across all test cases.
- ii. Average Mean Error: The mean absolute deviation between the interpolated generated signal and the reference signal, averaged across all test cases.
- iii. Average Maximum Error as % of the dynamic range: This corresponds to the error in (i.) as % of the maximum amplitude of the signal in the given frame, averaged across all test cases.
- iv. Average Mean Error as % of the dynamic range: This corresponds to the error in (ii.) as % of the maximum amplitude of the signal in the given frame, averaged across all test cases.

Additionally, the comparison logic excludes the final 20 sample points of the window from the error classification. These points exhibit minor transient artifacts in some test cases but since these points lie outside the region of interest of our system, they can be safely neglected to prevent skewing the accuracy metrics. Furthermore, these specific validation tests were conducted with the AWGN noise injection disabled to isolate deterministic algorithmic imprecision from stochastic noise components.

As shown in Figure (4.2) and in Tables (4.1) & (4.2), the generated data by the Virtual AFE shows consistent accuracy across all datasets. Hence, it can be used for simulation and verification purposes.

4. Implementation

	exp/contrast_speckle		exp/resolution_distorsion	
Metric	I	Q	I	Q
Avg Max Error	8.4×10^{-4}	8.9×10^{-4}	7.5×10^{-4}	8.1×10^{-4}
Avg Mean Error	1.6×10^{-5}	1.6×10^{-5}	1.5×10^{-5}	1.5×10^{-5}
Avg Max Error (% of DR)	0.67%	0.67%	0.53%	0.58%
Avg Mean Error (% of DR)	0.01%	0.01%	0.01%	0.01%

Table 4.1.: Virtual AFE Validation on the PICMUS experiments datasets

	in_vivo/carotid_cross		in_vivo/carotid_long	
Metric	I	Q	I	Q
Avg Max Error	9.5×10^{-4}	8.7×10^{-4}	9.4×10^{-4}	9.0×10^{-4}
Avg Mean Error	8.7×10^{-5}	8.7×10^{-5}	8.3×10^{-5}	8.4×10^{-5}
Avg Max Error (% of DR)	0.14%	0.13%	0.16%	0.16%
Avg Mean Error (% of DR)	0.01%	0.01%	0.01%	0.01%

Table 4.2.: Virtual AFE Validation on the PICMUS in_vivo datasets

4.2.4. Golden Model

To verify the functional correctness of the RTL implementation, a reference model was developed in Python (`complete_system_model.py`). The model combines high-precision floating-point arithmetic in some steps with bit-accurate integer operations in others, in a manner that reflects the numerical behavior of the hardware while avoiding unnecessary fixed-point complexity. The verification pipeline follows the structure of the hardware processing stages and selectively abstracts intermediate numerical representations where this does not affect functional equivalence:

1. Floating-Point Sparse DFT

The `streaming_dft_processor` function computes the DFT using floating-point arithmetic. In contrast to the RTL implementation, which incurs rounding and truncation effects due to fixed-point arithmetic at each accumulation stage, the reference model preserves ideal numerical precision at this stage.

2. Quantization via Logarithmic Power Conversion

A key observation enabling this modeling approach arises from the logarithmic power conversion stage. As mentioned in Equation (4.4), the power is approximated as:

$$P_{\text{dB}} \approx 3 \cdot \left\lfloor \log_2 \left(I^2 + Q^2 \right) \right\rfloor \quad (4.6)$$

This operation effectively functions as a quantization that dominates the small numerical discrepancies between the fixed-point DFT in hardware and the floating-point DFT in the reference model. As a result, although the intermediate complex accumulator values differ slightly, the resulting integer-valued power levels remain consistent, enabling reliable functional validation.

4. Implementation

3. Bit-Accurate Edge Detection

From the power computation stage onward, the reference model operates in a bit-accurate manner. It implements the same integer comparison logic and state machine behavior as the RTL in order to determine the bandwidth edge indices.

The outputs of the reference model are recorded as expected responses directly within the simulation vector files generated by `generate_vectors_top.py`. During the cycle-accurate simulation, the SystemVerilog testbench compares the acquired responses from the DUT with the corresponding expected responses.

Since the RTL module is not expected to achieve perfect numerical accuracy, the testbench allows controlled deviations under specific conditions:

- Within the region of interest, power values are allowed to deviate by a configurable threshold (e.g., 3 dB). This corresponds to a discrepancy of one in the integer logarithm. The region of interest consists of all the bins that have expected and acquired power levels above the noise floor.
- Bandwidth edge indices may differ by one bin. This tolerance accounts for cases where the power level at a given bin is close to or exactly equal to the absolute power threshold. In such scenarios, it is inconsequential whether the module outputs the current and the preceding bin, or the current and the subsequent bin. The linear interpolation will produce a value effectively at the correct edge. As a consequence, a bin index mismatch of one does not significantly affect the accuracy of the estimated bandwidth.
- For bins below the noise floor, deviations are allowed to be arbitrarily large, provided that both the expected and acquired power levels remain substantially below the absolute power threshold (e.g., by 10 dB). Errors in this region do not influence the identification of bandwidth edges and therefore do not compromise functional correctness.

All other mismatches between expected and acquired responses are recorded as errors. In addition to the comparison against the reference model inside the testbench itself, the acquired results are also exported to a CSV file, which allows an analysis script to calculate the deviations relative to the ground truth bandwidth edges as described in Section 4.2.1.

Results

5.1. Functional Accuracy

As briefly mentioned in Section 4.2.1, the functional accuracy of the proposed bandwidth estimation architecture was evaluated using four PICMUS benchmark datasets consisting of experimental phantoms and in vivo carotid acquisitions. The ground truth bandwidth edges were computed using a full FFT and were used as reference values to quantify the detection accuracy of the hardware estimates.

5.1.1. Error Metrics

The performance is characterized using three error metrics for both left and right bandwidth edges:

- **Mean Absolute Error (MAE)**
This metric describes the average magnitude of errors in the bandwidth estimation, expressed in MHz. It provides a direct measure of typical deviation from ground truth, independent of the absolute edge location.
- **Mean Absolute Percentage Error (MAPE)**
This corresponds to the average relative error normalized by the ground truth edge frequency, expressed as a percentage. MAPE enables a comparison independent from the modulation frequency of a transducer used in a specific dataset and instead captures the proportional accuracy of detection.
- **Standard Deviation (StdDev)**
The StdDev stands for the spread of estimation errors around the mean, indicating consistency. Similar to the MAE, it is also expressed in MHz.

5. Results

Metric	exp/contrast_speckle	exp/resolution_distorsion
Left Edge MAE	0.1021 MHz	0.1113 MHz
Left Edge MAPE	4.40%	4.86%
Left Edge StdDev	0.0564 MHz	0.0640 MHz
Right Edge MAE	0.1071 MHz	0.1207 MHz
Right Edge MAPE	4.77%	5.46%
Right Edge StdDev	0.0621 MHz	0.0899 MHz
Max Edge MAE	0.0990 MHz	0.1094 MHz
Max Edge MAPE	4.25%	4.75%
Decimation Safety	98.6%	100.0%

Table 5.1.: Functional accuracy evaluated on the PICMUS experiments datasets

Additionally, three system-level metrics characterize the end-to-end accuracy:

- **Maximum Edge MAE**
It quantifies the mean absolute frequency error of the maximum spectral edge (whichever of left or right has greater absolute frequency). This metric directly determines the accuracy of the acquired optimal decimation factor compared to the ground truth.
- **Maximum Edge MAPE**
This is completely analogous to the MAPE described before, but it uses the maximum spectral edge instead of just the right or the left one. It measures how close the acquired decimation factor is relatively to the ground truth one.
- **Decimation Safety**
This metric computes the percentage of test cases where the estimated decimation factor is less than or equal to the ground truth critical decimation factor. It quantifies the risk of aliasing due to bandwidth underestimation. A value approaching 100% indicates negligible probability of information loss. There is a tradeoff between the maximum edge MAE and the decimation safety, since the closer the acquired bandwidth estimates are to the ground truth ones, the larger the possibility for a bandwidth underestimation.

Tables 5.1 and 5.2 present the functional accuracy results for the PICMUS experimental phantom datasets and the PICMUS in vivo acquisitions respectively.

5.1.2. Performance on Experimental Datasets

The experimental phantom datasets (contrast_speckle and resolution_distorsion) represent controlled imaging scenarios with stationary scatterers and rather symmetric, well-behaved spectra. Performance on these datasets demonstrates the intrinsic accuracy of the sparse DFT approach under ideal conditions. Both left and right edge detection achieve MAPE values below 5.5%, with MAE ranging from around 0.10 to 0.12 MHz.

5. Results

Metric	in_vivo/carotid_cross	in_vivo/carotid_long
Left Edge MAE	0.1493 MHz	0.1210 MHz
Left Edge MAPE	6.92%	5.59%
Left Edge StdDev	0.1114 MHz	0.0818 MHz
Right Edge MAE	0.4625 MHz	0.4155 MHz
Right Edge MAPE	34.49%	27.00%
Right Edge StdDev	0.5159 MHz	0.4917 MHz
Max Edge MAE	0.1553 MHz	0.1256 MHz
Max Edge MAPE	7.19%	5.79%
Decimation Safety	99.6%	99.2%

Table 5.2.: Functional accuracy evaluated on the PICMUS in_vivo datasets

Critically, decimation safety exceeds 98.6% for both datasets. This near-perfect safety record demonstrates that the maximum edge estimate, which dictates the minimum safe sampling rate, is reliably conservative. Even in the cases where individual edge estimates deviate from ground truth, the maximum absolute frequency component remains correctly bounded, preventing aliasing-induced information loss.

5.1.3. Performance on In Vivo Datasets

The in vivo carotid datasets introduce significantly greater complexity due to physiological factors including blood flow, tissue motion, and the inherent asymmetry of Doppler spectra [21]. Left edge detection maintains strong performance with MAPE values of 6.92% and 5.59% for the cross-sectional and longitudinal views, respectively. MAE values (0.12–0.15 MHz) and standard deviations (0.08–0.11 MHz) remain comparable to the experimental datasets, demonstrating robust detection of the sharp spectral transition.

In contrast, right edge detection exhibits substantially degraded performance, with MAPE increasing to 27.00–34.49% and MAE rising to 0.42–0.46 MHz. Standard deviations exceed 0.49 MHz, indicating high sensitivity to local signal characteristics. This asymmetric performance directly reflects the physics of in vivo Doppler ultrasound: arterial blood flow generates asymmetric spectra with one sharp edge and one flatter, gradual edge. This phenomenon can be seen in Figure 5.1. Which of the sharp and gradual edges are on the left or the right side is determined by the direction of the blood-flow in this case.

Despite elevated right edge errors, the maximum edge MAPE remains below 7.2% for both in vivo datasets, with MAE constrained to 0.13–0.16 MHz. This solid max edge performance stems from the fact that the PICMUS in vivo carotid spectra are left-dominant, i.e., the maximum absolute frequency component typically resides at the sharp left edge rather than the gradual right edge. Consequently, decimation safety remains exceptionally high at 99.2–99.6%, confirming that the critical Nyquist criterion is robustly satisfied despite right edge detection challenges in these cases.

5. Results

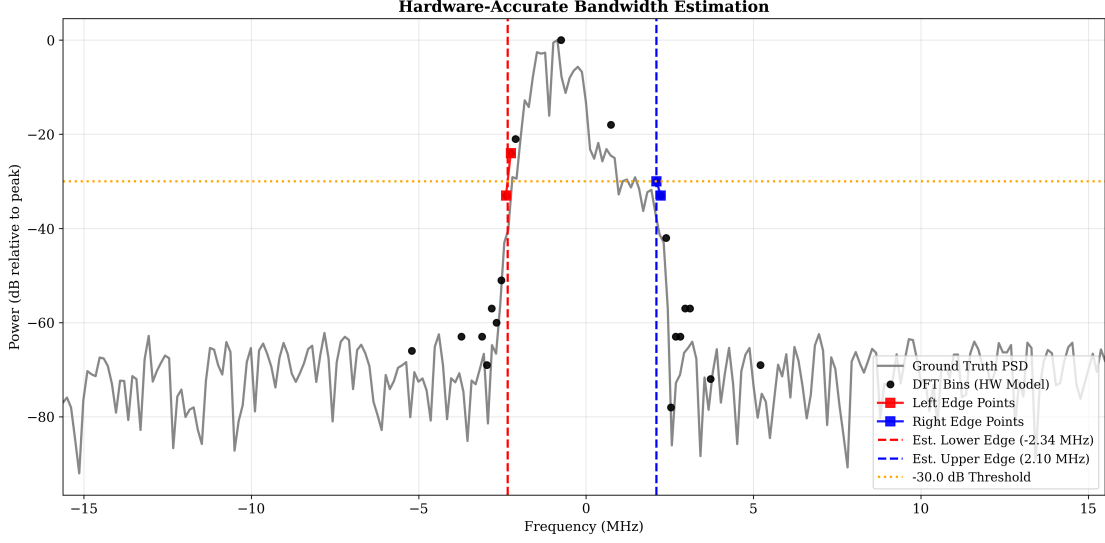


Figure 5.1.: Demonstration of the increased bandwidth overestimation for gradual spectra. The depicted frame corresponds to channel 37, window 15 of the in vivo carotid cross dataset.

5.1.4. Inherent Bandwidth Overestimation and Safety Margin

The proposed sparse DFT architecture exhibits a systematic tendency toward slight bandwidth overestimation, which operates as an intrinsic safety mechanism. This conservative bias arises from the fundamental limitation that sparse bin selection computes spectral power for only a subset of frequencies. Since the true spectral maximum may occur at any frequency within the full spectrum, the detected maximum across the sparse bin set always satisfies:

$$P_{max,sparse} \leq P_{max,gt} \quad (5.1)$$

When computing the absolute detection threshold as $P_{max} - 30$ dB, this inequality propagates to systematically yield a slightly lower absolute threshold as in the ground truth case.

$$T_{abs,sparse} = P_{max,sparse} - 30 \text{ dB} \leq P_{max,gt} - 30 \text{ dB} = T_{abs,gt} \quad (5.2)$$

Coming back to the in vivo results described in 5.1.3, this inherent threshold overestimation together with the linear interpolation explains the elevated right edge MAPE. It is clear that for flatter spectra, a threshold overestimation has a larger impact on the edge estimation than for steeper spectra. Looking at the linear interpolation formula

$$f^* = f_1 + (f_2 - f_1) \cdot \frac{T_{abs} - L_1}{L_2 - L_1}, \quad (5.3)$$

we can see that if the spectrum has a shallow slope ($L_2 - L_1$ is small), small errors in T_{abs} produce larger errors. This manifests primarily as right edge overestimation.

5. Results

However, as mentioned earlier, the proposed architecture successfully navigates this challenge by prioritizing the maximum edge, which is dominated by the sharp left side, for decimation ratio calculations.

In summary, a slightly lower absolute threshold causes the spectrum to cross the threshold detection boundary at frequencies further from the spectral peak, resulting in a small bandwidth overestimation. Importantly, overestimating the bandwidth produces conservative decimation factors that preserve signal content with margin, whereas underestimation risks catastrophic aliasing and irreversible data loss. The observed maximum edge MAE of 0.10–0.16 MHz translates to decimation safety margins well within acceptable system design tolerances, ensuring robust Nyquist compliance without excessive sampling overhead.

5.2. Achieved Data Reduction

The primary objective of the adaptive bandwidth estimation architecture is to enable aggressive decimation of the down-converted I/Q data stream while being as weakly lossy as possible. The baseline configuration employs a fixed decimation factor of $M = 4$, reducing the 125 MSPS ADC output to 31.25 MSPS per channel. This conservative choice ensures aliasing-free sampling for worst-case bandwidth but results in significant oversampling under typical imaging conditions.

A more informed comparison point is a naive worst-case decimation strategy based on transducer specifications. For the PICMUS datasets, which use a transducer with center frequency $f_c = 5.208$ MHz, the worst-case I/Q bandwidth spans $\pm f_c$ (accounting for the full relative bandwidth centered at baseband). With a $2\times$ oversampling safety factor, this yields a naive decimation of $M = \lfloor 125 / (4 \cdot f_c / 2) \rfloor = 12$. While this static approach already improves upon the baseline, it still has to assume worst-case spectral occupancy across all imaging depths and conditions.

The proposed adaptive approach eliminates this conservatism by adjusting decimation factors in real-time based on measured bandwidth, allowing the I/Q data to be compressed as much as the Nyquist theorem permits. Table 5.3 shows the achieved data reduction across four PICMUS datasets compared to both the baseline and naive decimation strategies.

The results demonstrate substantial and consistent data reduction across all datasets. Average decimation factors of $12.88\text{--}13.69\times$ represent a $3.2\text{--}3.4\times$ improvement over the $M = 4$ baseline, translating to 67–69% relative bandwidth savings. The adaptive system retains only 7.6–8.2% of the original ADC sample rate compared to the baseline’s 25%, discarding an additional 16.8–17.4% of the raw data stream while preserving full signal fidelity for downstream processing (>98.6% decimation safety from Sections 5.1.2 and 5.1.3).

5. Results

Dataset	Avg. M	Data Volume Kept	vs. Baseline (M=4)	vs. Naive (M=12)
Baseline (Fixed)	4.0	25.0%	—	—
Naive (Worst-case)	12.0	8.33%	$3.0\times$ (66.7%)	—
exp/contrast_speckle	13.56	7.4%	$3.39\times$ (70.5%)	$1.13\times$ (11.5%)
exp/resolution_distorsion	13.69	7.3%	$3.42\times$ (70.8%)	$1.14\times$ (12.3%)
in_vivo/carotid_cross	12.88	7.8%	$3.22\times$ (68.9%)	$1.07\times$ (6.8%)
in_vivo/carotid_long	13.07	7.6%	$3.27\times$ (69.4%)	$1.09\times$ (8.2%)

Table 5.3.: Data reduction performance across PICMUS datasets. "Data Volume Kept" indicates the percentage of the raw 125 MSPS ADC stream retained after decimation. Compression ratios and relative bandwidth savings (in parentheses) are shown versus both the fixed baseline ($M = 4$) and the naive worst-case approach ($M = 12$)

Compared to the naive worst-case decimation ($M = 12$), the adaptive approach achieves additional compression ratios of 1.07 – $1.14\times$, corresponding to 7–12% further bandwidth reduction. This gain represents the elimination of residual conservatism inherent in static worst-case assumptions. Critically, this additional compression is achieved without compromising signal integrity, the decimation safety margin is maintained across all scenarios. The adaptive system thus provides a solid balance between data reduction and fidelity, automatically exploiting signal-specific bandwidth characteristics that static strategies cannot capture.

5.3. Resource Utilization

The proposed hardware architecture was synthesized and implemented targeting the AMD Kria K26 System-on-Module (SoM), which is based on a Zynq UltraScale+ MPSoC (XCK26). This platform provides a resource budget of approximately 117k CLB Look-Up Tables (LUTs), 234k Flip-Flops (FFs), 1248 DSP slices, and 144 BRAM tiles.

Table 5.4 details the post-implementation resource of the complete bandwidth estimation top-level module and all of its constituent submodules, including the configuration loader, the spectral power estimator, and the edge detection logic. The hierarchical breakdown enables precise identification of resource costs at each architectural level, from the high-level system components down to individual computational blocks such as the CORDIC oscillators and logarithmic power converters

The complete design exhibits exceptional resource efficiency, occupying only 5.6% of the logic resources, 2.5% of the flip-flops, and approximately 1% of the available DSP slices on the Kria K26 SoM. Most critically, the implementation achieves zero BRAM utilization, which is frequently the scarcest resource in FPGA-based signal processing pipelines. BRAM often represents the bottleneck for buffering and clock-domain crossings between acquisition and host transmission stages. By employing a purely logic-based

5. Results

Resource	CLB LUTs		CLB Registers		CARRY8		F7 Muxes		F8 Muxes	
Total available	117'120		234'240		14'640		58'560		29'280	
top (TOTAL)	6515	5.56%	5805	2.48%	353	2.41%	644	1.10%	4	0.01%
bw edge detector	499	0.43%	468	0.20%	27	0.18%	72	0.12%	0	0.00%
calc threshold	0	0.00%	10	<0.01%	0	0.00%	0	0.00%	0	0.00%
find max	190	0.16%	390	0.17%	23	0.16%	0	0.00%	0	0.00%
find left edge	161	0.14%	34	0.01%	2	0.01%	40	0.07%	0	0.00%
find right edge	150	0.13%	34	0.01%	2	0.01%	32	0.05%	0	0.00%
config loader	198	0.17%	513	0.22%	0	0.00%	35	0.06%	0	0.00%
power estimator	5818	4.97%	4824	2.06%	326	2.23%	537	0.92%	4	0.01%
counter ctrl	161	0.14%	21	<0.01%	19	0.13%	0	0.00%	0	0.00%
dft accumulation	5573	4.76%	4540	1.94%	306	2.09%	537	0.92%	4	0.01%
osc bank	2507	2.14%	2397	1.02%	286	1.95%	1	<0.01%	0	0.00%
cordic	961	0.82%	992	0.42%	141	0.96%	0	0.00%	0	0.00%
log power tmux	84	0.07%	263	0.11%	1	<0.01%	0	0.00%	0	0.00%
log power	70	0.06%	19	<0.01%	1	<0.01%	0	0.00%	0	0.00%

Resource	CLB		LUT as Logic		LUT as Memory		BRAM Tiles		DSPs	
Total available	14'640		117'120		57'600		144		1248	
top (TOTAL)	1218	8.32%	6502	5.55%	13	0.02%	0	0.00%	12	0.96%
bw edge detector	116	0.79%	499	0.43%	0	0.00%	0	0.00%	0	0.00%
calc threshold	3	0.02%	0	0.00%	0	0.00%	0	0.00%	0	0.00%
find max	66	0.45%	190	0.16%	0	0.00%	0	0.00%	0	0.00%
find left edge	33	0.23%	161	0.14%	0	0.00%	0	0.00%	0	0.00%
find right edge	33	0.23%	150	0.13%	0	0.00%	0	0.00%	0	0.00%
config loader	114	0.78%	198	0.17%	0	0.00%	0	0.00%	0	0.00%
power estimator	1104	7.54%	5805	4.96%	13	0.02%	0	0.00%	12	0.96%
counter ctrl	34	0.23%	161	0.14%	0	0.00%	0	0.00%	0	0.00%
dft accumulation	1007	6.88%	5560	4.75%	13	0.02%	0	0.00%	10	0.80%
osc bank	469	3.20%	2494	2.13%	13	0.02%	0	0.00%	0	0.00%
cordic	177	1.21%	957	0.82%	4	<0.01%	0	0.00%	0	0.00%
log power tmux	102	0.70%	84	0.07%	0	0.00%	0	0.00%	2	0.16%
log power	27	0.18%	70	0.06%	0	0.00%	0	0.00%	2	0.16%

Table 5.4.: Complete resource utilization; configurable logic blocks (CLBs), look-up tables (LUTs), multiplexers (Muxes), block RAM (BRAM), and digital signal processing blocks (DSPs) utilization in absolute unites and as percentage of available resources on the Kria K26 SoM

5. Results

distributed memory approach with minimal storage requirements, the proposed Sparse DFT architecture preserves the entire BRAM capacity for other critical system tasks.

The final deployment target on the ListenToLight platform is a Zynq UltraScale+ ZU19EG FPGA, which offers substantially greater resources: 522,720 LUTs, 1,045,440 FFs, 1968 DSPs, and 984 BRAM tiles. On this target device, the proposed architecture would consume merely 1.2% of the available LUTs, 0.6% of the flip-flops, and 0.6% of the DSP slices, leaving plenty of headroom for multi-channel scaling and additional processing functionality.

5.3.1. Comparison with Industry-Standard FFT IP Cores

To evaluate the resource efficiency of the proposed Sparse DFT approach, a comparison was performed against industry-standard Xilinx FFT IP cores provided by Vivado. The comparison specifically targets streaming architectures, namely the Super Sample Rate (SSR) FFT implementations, which are designed for high-throughput applications such as the ListenToLight platform. Table 5.5 presents resource utilization for several representative FFT configurations compared against the proposed Sparse DFT.

Configuration	Transform Length	Fmax (MHz)	CLB LUTs	CLB Registers	DSPs	BRAM 18k	BRAM 36k
Sparse DFT	variable	375	5573	4540	10	0	0
kup_2_versal_dspcplx	512	604	27108	29496	180	0	0
kup_2_fixedp_ssr2_1024_scaling	1024	636	7101	7604	36	0	8
kup_2_fixedp_ssr4_1024_scaling	1024	636	9342	10539	48	0	8
kup_2_fixedp_ssr8_1024_scaling	1024	625	17651	21107	96	0	6
kup_2_fixedp_ssr64_1024_scaling	1024	533	95459	121816	528	0	36

Table 5.5.: Resource utilization comparison of the sparse DFT, proposed in this thesis, against streaming FFT IP cores for Kintex UltraScale+ FPGAs provided by Xilinx Vivado.

The comparison reveals two distinct advantages of the sparse approach:

- **Comparison with Zero-BRAM Architectures**

The configuration labeled `kup_2_versal_dspcplx` represents a high-performance 512-point FFT architecture that, like the proposed design, avoids BRAM usage entirely by utilizing distributed RAM (LUTRAM). However, to achieve this for a full spectral transform, it incurs a significant resource penalty. Compared to the proposed Sparse DFT, this zero-BRAM FFT consumes approximately 4.9x more LUTs, 6.5x more registers, and 18x more DSP slices. This substantial difference underscores the fundamental efficiency gain achieved by computing only the necessary bins rather than the complete spectrum.

5. Results

- **Comparison with SSR Streaming FFTs**

The SSR configurations represent standard solutions for high-throughput streaming applications. Even the most resource-efficient SSR variant (`ssr2`) consumes 1.3x more LUTs, 1.7x more registers, 3.6x more DSPs, and critically, requires 8 BRAM36k tiles. Higher-throughput SSR configurations scale resource usage even worse: The SSR64 variant consumes 17x more LUTs, 27x more registers, and 53x more DSPs, alongside 36 BRAM36k tiles. While these FFT architectures offer higher maximum clock frequencies (533-636 MHz vs 375 MHz for the Sparse DFT), this performance advantage comes at a prohibitive resource cost that is incompatible with massively parallel channel processing.

The key differentiator is BRAM consumption. As channel counts scale to support the 256-channel ListenToLight platform, BRAM availability becomes the primary system bottleneck, as previously mentioned. Traditional FFT-based approaches would require dedicated BRAM allocation per channel, rapidly exhausting available resources. In contrast, the proposed Sparse DFT's zero-BRAM design enables linear scaling limited only by logic fabric availability.

5.3.2. Scalability Analysis

The architecture is designed with modular scalability as a core principle. The overhead components, specifically the `config_loader` and the CORDIC oscillator phase calculations, represent a fixed cost that is instantiated once and shared across the different channels. Notably, the CORDIC oscillators constitute the single largest resource consumer within the Sparse DFT (approximately 45% of total LUTs and registers). Despite this, the complete system remains more efficient than the optimized industrial FFT implementations. This fixed-cost amortization becomes increasingly favorable as channel count grows: Duplicating only the DFT accumulation logic 3000 LUTs, 2100 Registers and 10 DSPs per channel, with no BRAM overhead.

For very high channel counts, it eventually becomes necessary to replicate the oscillator output registers to manage signal fanout constraints, but this represents a minor incremental cost compared to the savings from shared oscillator generation. This scalability ensures that the bandwidth estimation block remains a minimal contributor to the overall FPGA power and area budget, even in large-scale multi-channel systems. The zero-BRAM characteristic is particularly valuable in this context, as it eliminates the memory limitations that would otherwise constrain massive parallelization in traditional FFT-based designs.

5.4. Current Limitations and Future Work

The implemented architecture successfully validates the core sparse DFT concept for real-time bandwidth estimation, demonstrating substantial data reduction (Section 5.2) with robust functional accuracy (Section 5.1) while maintaining minimal resource footprint (Section 5.3). However, the current implementation incorporates two operational constraints that represent natural next steps toward full deployment.

As detailed in Section 4.1.2, the current timing controller requires continuous sample validity during the processing window. This burst-mode constraint arises from the decoupled, predictive control scheme of the CORDIC oscillators, which are triggered ahead of sample arrival to compensate for their 20-cycle pipeline latency. While this design choice works, full AXI-Stream compliance would enable more flexible integration with upstream processing blocks that may exhibit discontinuous sample validity behavior.

The current implementation is optimized for the baseline decimation factor of $M = 4$ (31.25 MSPS sample rate), corresponding to exactly 12 system clock cycles (at 375 MHz) per I/Q sample for the time-multiplexed DFT accumulation. While the bandwidth estimation logic itself supports arbitrary decimation factors, the timing controller presently requires the system to be in baseline decimation mode to calculate new decimation ratios.

Interestingly, both constraints admit a common solution through the addition of a shallow input elasticity buffer, as outlined in Section 4.1.2. A two-sample FIFO at the input interface would decouple oscillator phase generation from instantaneous sample validity, enabling dynamic gating of the enable signal for full AXI-Stream compliance. Simultaneously, this elasticity mechanism provides the infrastructure to support arbitrary integer decimation factors: since the 375 MHz system clock is an integer multiple of the 125 MSPS ADC rate, any integer decimation M yields an integer number of clock cycles per sample ($375 \text{ MHz} / [125 \text{ MSPS} / M]$). The pipeline can simply freeze after completing the 12-cycle time-multiplexed accumulation, holding state until the next sample arrives according to the current decimation ratio.

This straightforward extension path, requiring minimal additional logic, represents a natural evolution of the architecture toward deployment. The current burst-mode design successfully proves the fundamental concepts while providing a foundation for seamless extension to full streaming compliance and multi-rate operation.

Conclusion and Future Work

This thesis addressed the data throughput bottleneck in high-channel-count, high-sampling-rate ultrasound imaging systems within the context of the ListenToLight platform. To mitigate the discrepancy between massive data volumes generated during acquisition and limited host interface bandwidth, we proposed and implemented a closed-loop adaptive compression system that adjusts the decimation ratio based on real-time bandwidth estimation.

The core contribution is a sparse DFT architecture that enables efficient real-time spectral analysis without the memory overhead of traditional FFT-based approaches. By computing only 24 strategically selected frequency bins, the system identifies bandwidth edges through threshold crossing detection while maintaining a streaming processing paradigm. This architecture was realized using a deeply pipelined, time-multiplexed implementation targeting a Xilinx Zynq UltraScale+ FPGA operating at 375 MHz.

The system incorporates a decimation filter with real-time adjustable gain and cutoff frequency. It was demonstrated in simulation that using a filter with edge-boosting characteristics preserves signal fidelity at the transducer's bandwidth edges. On synthetic data, this achieves a SINAD improvement of +4.92 dB and an SQNR gain of +11.88 dB compared to generic LPF approaches, translating to an effective resolution improvement of +1.97 bits by maximizing dynamic range utilization prior to re-quantization.

Validation of the real-time bandwidth estimation across four PICMUS benchmark datasets demonstrates substantial performance gains. The system achieves compression ratios of $3.22\times$ to $3.42\times$ relative to the $M = 4$ baseline, corresponding to 68–70% bandwidth savings. Compared to a naive worst-case decimation strategy ($M = 12$) based on transducer specifications, the adaptive approach achieves additional compression ratios of 1.07 – $1.14\times$, representing 7–12% further bandwidth reduction by eliminating residual static conservatism. Critically, decimation safety exceeds 98.6% across all datasets, with

6. Conclusion and Future Work

in vivo scenarios achieving 99.2–99.6%, indicating negligible aliasing risk. Functional accuracy shows maximum edge MAPE below 7.2%, with experimental datasets achieving values as low as 4.25%.

The implementation exhibits remarkable resource efficiency: 5.6% of available LUTs, 2.5% of flip-flops, 1% of DSP slices, and critically, zero BRAM utilization on the Kria K26 SoM. Compared to industry-standard Xilinx FFT IP cores, equivalent zero-BRAM FFT implementations consume $4.9\times$ more LUTs, $6.5\times$ more registers, and $18\times$ more DSP slices, while streaming FFT variants additionally require 8–36 BRAM tiles. This BRAM-free design enables linear scaling of channel count limited only by logic fabric availability, crucial for the 256-channel ListenToLight platform.

In summary, this work validates the sparse DFT approach for real-time bandwidth estimation in streaming ultrasound applications, achieving high compression efficiency while maintaining signal fidelity and preserving resources necessary for large-scale multi-channel deployments.

A natural extension of this work would be the integration of a shallow input elasticity buffer to enhance interface flexibility. By adding a two-sample FIFO, the design can be upgraded to support discontinuous AXI validity signals and enable bandwidth estimation across dynamic decimation rates, effectively bridging the gap to full AXI-Stream compliance and multi-rate support simultaneously.

Beyond these immediate refinements, the iterative refinement strategy mentioned in Section 3.1, using detected edges to re-center search clusters and increase window length, represents a promising avenue for near-optimal bandwidth tracking. Finally, the architecture is designed to facilitate extension into multi-channel parallel processing; future implementations could thus exploit shared oscillator generation to significantly amortize fixed costs.

Signal Processing Derivations

A.1. Further Details on I/Q Demodulation

Starting point for this derivation is the digitized RF signal $x_{RF}(t)$ in time domain. After mixing this signal with a complex sinusoid with frequency $f_{demod} \approx f_c$, corresponding to the transducer center frequency, we get the following expression:

$$x_{mixed}(t) = x_{RF}(t) \cdot e^{-j2\pi f_c t} = x_{RF}(t) \cdot (\cos(2\pi f_c t) - j \cdot \sin(2\pi f_c t)) \quad (A.1)$$

In hardware, both the in-phase (I) and the quadrature-phase (Q) are calculated independently. Hence, we get:

$$\begin{aligned} \tilde{x}_I(t) &= \mathcal{R}\{x_{mixed}(t)\} = x_{RF}(t) \cdot \cos(2\pi f_c t) \\ \tilde{x}_Q(t) &= \mathcal{I}\{x_{mixed}(t)\} = -x_{RF}(t) \cdot \sin(2\pi f_c t) \end{aligned} \quad (A.2)$$

Let us analyze the spectral content of the in-phase signal $\tilde{X}_I(f)$.

$$\begin{aligned} \tilde{X}_I(f) &= \mathcal{F}\{x_{RF}(t) \cdot \cos(2\pi f_c t)\}(f) \\ &= X_{RF}(f) * \frac{1}{2}(\delta(f + f_c) + \delta(f - f_c)) \\ &= \underbrace{\frac{1}{2}X_{RF}(f + f_c)}_{\text{Shift from } -f_c \text{ to DC}} + \underbrace{\frac{1}{2}X_{RF}(f - f_c)}_{\text{Shift from } +f_c \text{ to DC}} \end{aligned} \quad (A.3)$$

The operation effectively shifts the spectral components originally centered at $-f_c$ and $+f_c$ to the baseband (around $f = 0$). Since $x_{RF}(t)$ is real-valued, its spectrum exhibits Hermitian symmetry, defined as $X_{RF}(-f) = X_{RF}^*(f)$.

A. Signal Processing Derivations

Focusing on the baseband region (frequencies $f \approx 0$) after low-pass filtering, the resulting spectrum $X_I(f)$ is the superposition of the shifted positive and negative frequency components:

$$X_I(f) \approx \frac{1}{2} (X_{RF}^*(f) + X_{RF}(f)) = \mathcal{R}\{X_{RF}(f)\} \quad (\text{A.4})$$

Thus, the in-phase component captures the real part of the complex spectral envelope.

Similarly, we derive the spectrum for the quadrature signal $\tilde{x}_Q(t)$. The time-domain multiplication with $-\sin(2\pi f_c t)$ corresponds to a convolution in the frequency domain with $\frac{j}{2}(\delta(f - f_c) - \delta(f + f_c))$.

$$\begin{aligned} \tilde{X}_Q(f) &= X_{RF}(f) * \frac{j}{2} (\delta(f - f_c) - \delta(f + f_c)) \\ &= \frac{j}{2} X_{RF}(f - f_c) - \frac{j}{2} X_{RF}(f + f_c) \end{aligned} \quad (\text{A.5})$$

Again, considering the baseband region ($f \approx 0$) after low-pass filtering, the term $X_{RF}(f - f_c)$ shifts the negative spectral component ($X_{RF}(-f_c)$) to DC, while $X_{RF}(f + f_c)$ shifts the positive component ($X_{RF}(f_c)$) to DC. Applying Hermitian symmetry $X_{RF}(-f) = X_{RF}^*(f)$:

$$\begin{aligned} X_Q(f) &\approx \frac{j}{2} (X_{RF}(-f) - X_{RF}(f)) \\ &= \frac{j}{2} (X_{RF}^*(f) - X_{RF}(f)) \end{aligned} \quad (\text{A.6})$$

Recall that for any complex number z , $z^* - z = -2j \cdot \mathcal{I}\{z\}$. Therefore:

$$X_Q(f) = \frac{j}{2} (-2j \cdot \mathcal{I}\{X_{RF}(f)\}) = \mathcal{I}\{X_{RF}(f)\} \quad (\text{A.7})$$

This derivation demonstrates that the I/Q demodulation process effectively decomposes the symmetric RF spectrum into its real and imaginary spectral components at baseband.

A.2. On Signal-to-Quantization-Noise Ratios

For a uniform quantizer with step size Δ , the quantization error is uniformly distributed between $-\frac{\Delta}{2}$ and $+\frac{\Delta}{2}$. According to standard literature, the variance (quantization noise power) is given by:

$$P_n = \frac{\Delta^2}{12} \quad (\text{A.8})$$

A. Signal Processing Derivations

For a full scale sine wave with amplitude A , the signal power is:

$$P_s = \frac{A^2}{2} \quad (\text{A.9})$$

An N -bit ADC spanning a range of $2A$, i.e., from $-A$ to $+A$, has the following relationship between step size, range and number of bits:

$$\Delta = \frac{2A}{2^N} \quad (\text{A.10})$$

Combining equations (A.8)-(A.10), we get the following expression for the SQNR:

$$\text{SQNR} = \frac{P_s}{P_n} = \frac{A^2/2}{\Delta^2/12} = \frac{6A^2}{\Delta^2} = \frac{6A^2}{\left(\frac{2A}{2^N}\right)^2} = 1.5 \cdot 2^{2N} \quad (\text{A.11})$$

Converting to dB yields:

$$\begin{aligned} \text{SQNR}_{\text{dB}} &= 10\log_{10}\left(1.5 \cdot 2^{2N}\right) = 10\log_{10}(1.5) + 10\log_{10}\left(2^{2N}\right) \\ &\approx 1.76 + 20 \cdot N \cdot \log_{10}(2) \approx 1.76 + 6.02 \cdot N \end{aligned} \quad (\text{A.12})$$

Appendix	B
----------	----------

Task Description

**BACHELOR THESIS AT THE DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL
ENGINEERING**

FALL SEMESTER 2025

Matteo Dietz

**Development of a reconfigurable active filtering
streaming processing block for ListenToLight**

September 1, 2025

Advisors: Federico Villani, ETZ J69.2, villanif@ethz.ch
Dr. Christian Vogt, ETZ D97.7, christian.vogt@pbl.ee.ethz.ch
Dr. Andrea Cossettini, OAT U27, cosandre@iis.ee.ethz.ch
Supervisor: Prof. Dr. Luca Benini, lbenini@iis.ee.ethz.ch
Handout: 15 September 2025
Due: TBD

The final report will be submitted in electronic format. All copies remain property of the Integrated Systems Laboratory.

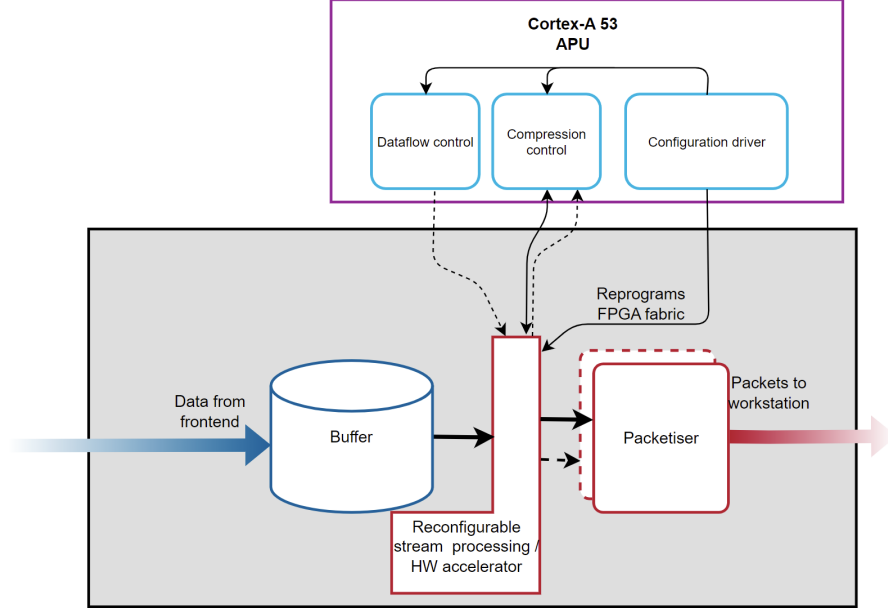


Figure 1: High-level architectural view of the FPGA reconfigurable section.

1 Introduction

Most open ultrasound (US) systems use FPGAs as the core processing, data-transport, and control units [boniUltrasoundOpenPlatforms2018] because they provide the speed and flexibility needed to manage the large number of high-speed digital connections required for software-defined US front-ends. However, in open US systems the FPGA has mostly been limited to data transfer and control rather than processing.

There have been attempts at on-system processing, but these are typically constrained in performance or channel count and tied to a specific imaging modality, limiting the flexibility of open systems [kimSingleFPGA-basedPortable2012, wuFPGABasedBackendSystem2019]. Despite growing adoption in the research community, many US platforms are limited by host-interface bandwidth; the host’s lower inherent parallelism makes this link a bottleneck even for advanced systems. Initial data reduction can be performed on advanced AFEs, such as the TI AFE58JD48 used in the ListenToLight system, which provides an I/Q downsampling and demodulation block. While transmitting I/Q-demodulated data to the FPGA reduces data volume, variability in data encoding, downsampling configuration, and transmission frequency complicates integration into the FPGA acquisition and processing pipeline. For instance, a naïve implementation would require separate FPGA pipelines and decoders for baseband and I/Q data, increasing system complexity.

US acquisition—driven by biological signals and human-machine interaction—must be flexible and adaptable to the application. One practical approach is to reserve a reconfigurable FPGA region that tailors processing to the specific use case.

Optoacoustic (OA) imaging has demonstrated significant potential in biomedical research and is increasingly used in preclinical and clinical applications. In contrast to US imaging, OA relies on the generation of ultrasound by absorption of nanosecond light pulses in tissue. Since acoustic scattering in tissue is weaker than optical scattering, OA can probe optically dense tissues while providing rich optical contrast across different penetration scales.

In general, OA and US share key advantages for biomedical applications, including non-ionising radiation, real-time imaging capability, high spatial resolution, and portability. The two modalities provide complementary information: US offers anatomical and structural detail, while OA adds functional contrast, improving clinical applicability. Because both modalities detect US waves, the electronics used in conventional pulse-echo US systems can be repurposed for OA data acquisition. However, conventional B-mode strategies differ from the tomographic approaches that yield optimal OA image quality.

2 Project Description

2.1 Task description

To achieve the goal of showing the feasibility and high processing power of on-FPGA reconfigurable streaming processing, the following tasks should be accomplished:

- **Task I: Literature review** Conduct a concise review of stream processing, FPGA (re)configuration, and ultrasound data processing. The goal is to select a preferred strategy and choose a streaming algorithm for demonstration.
- **Task II: Familiarization with the existing system; initial I/Q acquisition** Use the existing AFE→FPGA testboard to acquire test data. Program the AFE with the evaluation-board software, set up the FPGA ILA for capture, and validate data transfer from AFE to FPGA.
- **Task III: Basic reordering and packaging of I/Q data** Implement on-FPGA reordering and packaging for I/Q data. The block leverages AFE configuration to correctly reorder and packetize the streamed data for downstream processing, forming the basis for subsequent tasks.
- **Task IV: Design of a streaming processing algorithm** Design and implement a streaming algorithm in HDL or HLS. Depending on the literature review and the student's interest, candidates include compression, denoising, or filtering. The objective is to demonstrate that on-FPGA streaming improves data quality and/or usability under real-time constraints. If suitable candidates are found, advanced, novel algorithms are preferred.

- **Task V: Reconfiguration proof-of-concept (optional)** Demonstrate run-time FPGA reconfiguration with a short, bounded interruption of the stream. The objective is to switch the streaming pipeline between I/Q-demodulated and baseband variants (and vice versa).
- **Task VI: Report and presentation** Prepare the final written report and the group presentation.

3 Milestones

1. **M1 (Task I):** Key design requirements identified from the literature; target streaming algorithm and evaluation metrics selected.
2. **M2 (Task II):** AFE→FPGA data path validated; I/Q test dataset captured; interfaces and capture workflow defined.
3. **M3 (Task III):** I/Q reordering and packaging block implemented and verified.
4. **M4 (Task IV):** Streaming processing block implemented on FPGA and demonstrated at target throughput/latency.
5. **M5 (Task V, optional):** Run-time reconfiguration demonstrated.
6. **M6 (Task VI):** Final presentation delivered and report submitted.

4 Project Realization

4.1 Project Plan

Within the first week of the project, you will be asked to prepare a project plan. This plan should identify the tasks to be performed during the project and sets deadlines for those tasks. The prepared plan will be a topic of discussion of the first week's meeting between you and your advisers. Note that the project plan should be updated constantly depending on the project's status.

4.2 Meetings

Weekly meetings and reports must be held. The exact time and location of these meetings will be determined within the first week of the project in order to fit the student's and the assistant's schedule. These meetings will be used to evaluate the status and progress of the project. Besides these regular meetings, additional meetings can be organized to address urgent issues as well.

4.3 Report

Documentation is an important and often overlooked aspect of engineering. One final report has to be completed within this project. The common language of engineering is de facto English. Therefore, the final report of the work is preferred to be written in English. Any form of word processing software is allowed for writing the reports, nevertheless, the use of L^AT_EX with Tgif¹ or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

Final Report The final report has to be presented at the end of the project and a digital copy needs to be handed in. Note that this task description is part of your report and has to be attached to your final report.

4.4 Presentation

There will be a presentation (15 min for semester and bachelor theses, and 20 min for the MS thesis presentation followed by 5 min Q&A) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

References

- [1] “59355 - Vivado IP Flows - How to use one Block Design inside another Block Design.” (), [Online]. Available: https://support.xilinx.com/s/article/59355?language=en_US (visited on 05/27/2023).
- [2] H. Ahangari, M. M. Özdal, and Ö. Öztürk, “HLS-based High-throughput and Work-efficient Synthesizable Graph Processing Template Pipeline,” *ACM Transactions on Embedded Computing Systems*, vol. 22, no. 2, 34:1–34:24, Jan. 24, 2023, ISSN: 1539-9087. DOI: 10.1145/3529256. [Online]. Available: <https://dl.acm.org/doi/10.1145/3529256> (visited on 12/05/2023).
- [3] J. Chen, A. C. H. Yu, and H. K.-H. So, “Design considerations of real-time adaptive beamformer for medical ultrasound research using FPGA and GPU,” in *2012 International Conference on Field-Programmable Technology*, Dec. 2012, pp. 198–205. DOI: 10.1109/FPT.2012.6412134.
- [4] “Dh0017-vivado-partial-reconfiguration-hub.html • Viewer • AMD Adaptive Computing Documentation Portal.” (), [Online]. Available: <https://docs.xilinx.com/v/u/en-US/dh0017-vivado-partial-reconfiguration-hub> (visited on 12/05/2023).

¹See: <http://bourbon.usc.edu:8001/tgif/index.html> and <http://www.dz.ee.ethz.ch/en/information/how-to/drawing-schematics.html>.

- [5] M. d. A. Freitas, M. R. Jimenez, H. Benincaza, and J. P. von der Weid, “A new lossy compression algorithm for ultrasound signals,” in *2008 IEEE Ultrasonics Symposium*, Nov. 2008, pp. 1885–1888. DOI: 10.1109/ULTSYM.2008.0464.
- [6] D. L. N. Hettiarachchi and E. J. Balster, “Fixed-Point Processing of the SAR Back-Projection Algorithm on FPGA,” *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 14, pp. 10 889–10 902, 2021, ISSN: 2151-1535. DOI: 10.1109/JSTARS.2021.3119007.
- [7] M. G. Jordan, G. Korol, T. Knorst, M. B. Rutzig, and A. C. S. Beck, “Energy-aware fully-adaptive resource provisioning in collaborative CPU-FPGA cloud environments,” *Journal of Parallel and Distributed Computing*, vol. 176, pp. 55–69, Jun. 1, 2023, ISSN: 0743-7315. DOI: 10.1016/j.jpdc.2023.02.009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731523000308> (visited on 04/19/2023).
- [8] G.-D. Kim, C. Yoon, S.-B. Kye, *et al.*, “A single FPGA-based portable ultrasound imaging system for point-of-care applications,” *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 59, no. 7, pp. 1386–1394, Jul. 2012, ISSN: 1525-8955. DOI: 10.1109/TUFFC.2012.2339.
- [9] “Method 1: Add and Link Files • Vivado Design Suite User Guide: Dynamic Function eXchange (UG909) • Reader • AMD Adaptive Computing Documentation Portal.” (), [Online]. Available: <https://docs.xilinx.com/r/en-US/ug909-vivado-partial-reconfiguration/Method-1-Add-and-Link-Files> (visited on 12/05/2023).
- [10] A. Özbek, X. L. Deán-Ben, and D. Razansky, “Compressed Optoacoustic Sensing of Volumetric Cardiac Motion,” *IEEE Transactions on Medical Imaging*, vol. 39, no. 10, pp. 3250–3255, Oct. 2020, ISSN: 1558-254X. DOI: 10.1109/TMI.2020.2985134. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9102986> (visited on 02/02/2024).
- [11] A. Ozbek, X. L. Deán-Ben, and D. Razansky, “Realtime parallel back-projection algorithm for three-dimensional optoacoustic imaging devices,” presented at the European Conferences on Biomedical Optics, V. Ntziachristos and C. P. Lin, Eds., Munich, Germany, Jun. 14, 2013, p. 88000I. DOI: 10.1117/12.2033376. [Online]. Available: <http://proceedings.spiedigitallibrary.org/proceeding.aspx?doi=10.1117/12.2033376> (visited on 02/02/2024).
- [12] A. Özbek, X. L. Deán-Ben, and D. Razansky, “Universal Real-Time Adaptive Signal Compression for High-Frame-Rate Optoacoustic Tomography,” *IEEE Transactions on Medical Imaging*, vol. 41, no. 10, pp. 2903–2911, Oct. 2022, ISSN: 1558-254X. DOI: 10.1109/TMI.2022.3175471. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9778277> (visited on 02/02/2024).
- [13] A. Sarioglou, “Data Reduction of Ultrasound Signals on FPGA,”

- [14] J. Shi and D. Rakhmatov, “Fixed-Point CPWC Ultrasound Image Reconstruction,” in *2019 IEEE International Ultrasonics Symposium (IUS)*, Oct. 2019, pp. 1282–1285. DOI: 10.1109/ULTSYM.2019.8926043.
- [15] “Using HLS Streams • Vitis High-Level Synthesis User Guide (UG1399) • Reader • AMD Adaptive Computing Documentation Portal.” (), [Online]. Available: <https://docs.xilinx.com/r/en-US/ug1399-vitis-hls/Using-HLS-Streams> (visited on 12/05/2023).
- [16] Y. Wang, Y. Zhou, Q. S. Wang, *et al.*, “Developing medical ultrasound beamforming application on GPU and FPGA using oneAPI,” in *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, Jun. 2021, pp. 360–370. DOI: 10.1109/IPDPSW52791.2021.00064.
- [17] S. Wendel, A. Geiger, E. Liegmann, *et al.*, “UltraZohm - a Powerful Real-Time Computation Platform for MPC and Multi-Level Inverters,” in *2019 IEEE International Symposium on Predictive Control of Electrical Drives and Power Electronics (PRECEDE)*, May 2019, pp. 1–6. DOI: 10.1109/PRECEDE.2019.8753306.

Zurich, September 1, 2025

Prof. Dr. Luca Benini

Bibliography

- [1] O. Drori, A. Mamistvalov, O. Solomon, and Y. C. Eldar, "Compressed ultrasound imaging: from sub-nyquist rates to super-resolution," 2021. [Online]. Available: <https://arxiv.org/abs/2107.11263>
- [2] N. Nyayapathi, E. Zheng, Q. Zhou, M. Doyley, and J. Xia, "Dual-modal photoacoustic and ultrasound imaging: from preclinical to clinical applications," *Frontiers in photonics*, vol. 5, 02 2024.
- [3] N. Nagornov, P. Lyakhov, M. Valueva, and M. Bergerman, "Rns-based fpga accelerators for high-quality 3d medical image wavelet processing using scaled filter coefficients," *IEEE Access*, vol. 10, pp. 1–1, 01 2022.
- [4] J. Kim, S. Yeo, J. Kim, K. Kim, T.-K. Song, C. Yoon, and J. Sung, "Real-time lossless compression algorithm for ultrasound data using bl universal code," *Sensors*, vol. 18, p. 3314, 10 2018.
- [5] T. Kim, C. Lee, J. jun Kim, and T. kyong Song, "A new dynamic decimation filter using polyphase macs for medical ultrasound imaging," in *2008 IEEE Ultrasonics Symposium*, 2008, pp. 1394–1396.
- [6] H. Kang, J. Kang, and T.-K. Song, "Generalised dynamic decimation method using polyphase macs for ultrasound imaging," *Electronics Letters*, vol. 51, pp. 451–452, 03 2015.
- [7] C. Poggi and M. Palavecino, "Ultrasound principles and instrumentation," *Surgery Open Science*, vol. 18, pp. 123–128, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2589845024000241>
- [8] J. Kirkhorn, "Introduction to IQ Demodulation of RF Data," Sep. 1999, technical note. [Online]. Available: <https://htorp.folk.ntnu.no/Undervisning/TTK10/IQdemodulation.pdf>
- [9] "AFE58JD48: 12.8-GB JESD204B ultrasound AFE with 16-bit 125-MSPS ADC," Datasheet, Texas Instruments. [Online]. Available: <https://www.ti.com/product/AFE58JD48>

Bibliography

- [10] S. Scholl, "Exact Signal Measurements Using FFT Analysis," Microelectronic Systems Design Research Group, Technical University of Kaiserslautern, Kaiserslautern, Germany, Tech. Rep., 2016. [Online]. Available: https://kluedo.ub.rptu.de/frontdoor/deliver/index/docId/4293/file/exact_fft_measurements.pdf
- [11] F. Harris, "On the use of windows for harmonic analysis with the discrete fourier transform," *Proceedings of the IEEE*, vol. 66, no. 1, pp. 51–83, 1978.
- [12] Mathuranathan. (2020) Equivalent noise bandwidth (ENBW) of window functions. GaussianWaves. [Online]. Available: <https://www.gaussianwaves.com/2020/09/equivalent-noise-bandwidth-enbw-of-window-functions/>
- [13] V. Havin and B. Jöricke, *The Uncertainty Principle in Harmonic Analysis*, 1st ed., ser. Ergebnisse der Mathematik und ihrer Grenzgebiete. 3. Folge / A Series of Modern Surveys in Mathematics. Springer-Verlag Berlin Heidelberg, 1994. [Online]. Available: <https://doi.org/10.1007/978-3-642-78377-7>
- [14] K. Dubey, "The Fourier Uncertainty Principles," 2021. [Online]. Available: <https://math.uchicago.edu/~may/REU2021/REUPapers/Dubey.pdf>
- [15] M. Leiber, "Adaptive time-frequency analysis and data normalization: contributions to monitoring under varying conditions," Doctoral Thesis, Ecole Normale Supérieure (ENS), 2024, submitted to HAL on 16 Jan 2025. [Online]. Available: <https://hal.science/tel-04892428v1>
- [16] AMD/Xilinx, *CORDIC v6.0 LogiCORE IP Product Guide, Vivado Design Suite (PG105)*, AMD, Aug. 2021, version 6.0. [Online]. Available: <https://docs.amd.com/v/u/en-US/pg105-cordic>
- [17] H. Liebgott, A. Rodriguez-Molares, F. Cervenansky, J. Jensen, and O. Bernard, "Plane-wave imaging challenge in medical ultrasound," in *2016 IEEE International Ultrasonics Symposium (IUS)*, 2016, pp. 1–4.
- [18] S. H. Thon, R. E. Hansen, and A. Austeng, "Point detection in ultrasound using prewhitening and multilook optimization," *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 69, no. 6, pp. 2085–2097, 2022.
- [19] (2024) Arbitrary Magnitude Filter Design. MathWorks. [Online]. Available: <https://www.mathworks.com/help/dsp/ug/arbitrary-magnitude-filter-design.html>
- [20] "IEEE Standard for Terminology and Test Methods for Analog-to-Digital Converters," *IEEE Std 1241-2010 (Revision of IEEE Std 1241-2000)*, pp. 1–139, 2011.
- [21] L. Y. Mo and R. Cobbold, "'Speckle' in Continuous Wave Doppler Ultrasound Spectra: A Simulation Study," *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, vol. 33, no. 6, pp. 747–753, 1986.